

Dimitar Minchev

Developing Cross Platform Apps

Published
with GitBook



Table of Contents

Introduction	1.1
Chapter 1. UWP	1.2
History	1.2.1
Mobile Devices	1.2.1.1
Universal Windows Platform	1.2.1.2
Installation	1.2.2
Enabling Developer Mode	1.2.2.1
Registering as an App Developer	1.2.2.2
How to check the operating system version?	1.2.2.3
Applications	1.2.3
Suma 1.0	1.2.3.1
Fibonacci 1.0	1.2.3.2
Primes 1.0	1.2.3.3
Phone Book 1.0	1.2.3.4
Clicker Mania 1.1	1.2.3.5
Clicker Mania 1.2	1.2.3.6
HTML Downloader 1.0	1.2.3.7
RSS Reader 1.0	1.2.3.8
JSON Reader 1.0	1.2.3.9
Chapter 2. Xamarin.Forms	1.3
History	1.3.1
Applications	1.3.2
Suma 2.0	1.3.2.1
Fibonacci 2.0	1.3.2.2
Primes 2.0	1.3.2.3
Phone Book 2.0	1.3.2.4
Clicker Mania 2.0	1.3.2.5
HTML Downloader 2.0	1.3.2.6
RSS Reader 2.0	1.3.2.7
JSON Reader 2.0	1.3.2.8
Chapter 3. WinUI	1.4
History	1.4.1
Installation	1.4.2
Applications	1.4.3
Suma 3.0	1.4.3.1
Chapter 4. MAUI	1.5

History	1.5.1
Installation	1.5.2
Applications	1.5.3
Suma 4.0	1.5.3.1
Conclusion	1.6
Note	1.6.1
Book	1.6.2
Author	1.6.3
License	1.6.4
Format	1.6.5

Developing Cross-Platform Apps

This project **Developing Cross-Platform Apps** presents Microsoft technologies for developing cross-platform applications. The tutorial is available to read for free as an [e-book](#). The learning resources are available for free download from the project's [repository](#). All materials are distributed under the [CC-BY-NC-SA](#) license.

Format	ISBN
PDF	978-619-7126-66-2
MOBI	978-619-7126-67-9
EPUB	978-619-7126-68-6

Author

Dimitar Minchev is a university lecturer at the Faculty of Informatics and Technical Sciences at Burgas Free University. He prepares students for the [Republican Student Programming Olympiad](#) through the [Competitive Programming Club](#). He organizes a competition for desktop and mobile application development: [HACKATHON @ BFU](#). He founded the [Programming Talents Academy](#) and the [Robotics School](#) for school students in Burgas. He initiated the student programming contest [CODE@BURGAS](#). He participates in the national program [Training for IT Skills and Career](#) of the Ministry of Education and Science.

Contact
Work: tel. +359 56 900 477 and e-mail: mitko@bfu.bg
Personal: mobile +359 899 148 872 and e-mail: dimitar.minchev@gmail.com
Blog: http://www.minchev.eu

Chapter 1. UWP

Info

This chapter introduces the reader to the exciting world of Microsoft technologies for developing **Universal Windows Platform (UWP)** applications.

Tip

I wish you lots of fun, failures, and successes while using Microsoft technologies for designing and developing cross-platform applications.

Note

The examples in this section were developed using the following versions of the operating system and the development environment:

- Microsoft Windows 11 Version 10.0.22000.282
- Microsoft Visual Studio Community 2022 Version 17.0.0

History

Microsoft is a multinational company operating in the field of computer technology and software development. Its headquarters are located in Redmond, Washington, United States. It was founded by Bill Gates and Paul Allen in 1975.



Fig. 1.1. Paul Allen (left) and Bill Gates (right) founding Microsoft in 1975

The first version of the MS-DOS operating system was released in August 1981, and the graphical interface Windows 1.0 on November 20, 1985. The first version of the popular Microsoft Office productivity suite was released on November 19, 1990.

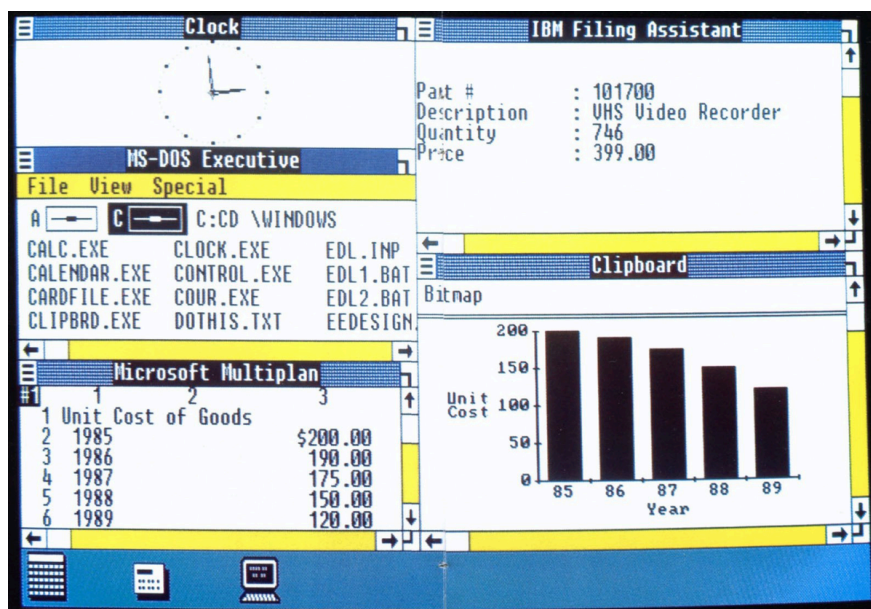


Fig. 1.2. Microsoft Windows 1.0 released in 1985

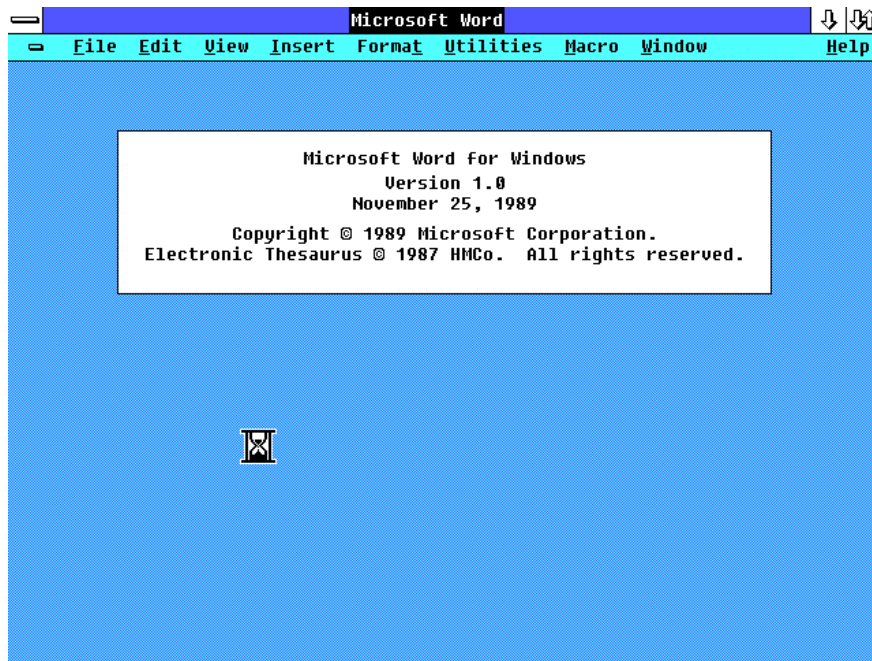


Fig. 1.3. Microsoft Word 1.0 released in 1989

The Windows 8 and 8.1 operating systems were introduced on August 1, 2012 and August 27, 2013 respectively. Support for Windows 8 and 8.1 ended on January 12, 2016 and January 10, 2023 respectively. The latest major release of the widely used desktop operating system, Windows 10, was released on July 15, 2015.



Fig. 1.4. Windows 8.1 released in 2013



Fig. 1.5. Windows 10 released in 2015

According to official figures published by Microsoft, installations of Windows 10 and 11 for the period from 2015 to 2025 are shown in Table 1.

Month and year	Number of devices
July 2015	Windows 10 launch
September 2016	400 million (Windows 10)
March 2017	500 million (Windows 10)
September 2018	700 million (Windows 10)
March 2020	1 billion (Windows 10)
October 2021	Windows 11 launch
October 2023	400 million (Windows 11)
April 2022	1.4 billion (total Windows 10 and 11)
July 2025	Windows 11 – 52% market share, Windows 10 – 44.59%

Table 1 Installations of the Windows 10 and 11 operating systems

According to official figures published by Microsoft, the public releases of the Windows 10 operating system are shown in Table 2.

Version	Build	Release
1507	10240	July 2015
1511	10586	November 2015
1607	14393	August 2016
1703	15063	April 2017
1709	16299	October 2017
1803	17134	April 2018
1809	17763	October 2018
1903	18362	May 2019
1909	18363	November 2019
2004	19041	May 2020
20H2	19042	October 2020
21H1	19043	May 2021
21H2	19044	November 2021
22H2	19045	October 2022

Table 2 Public releases of the Windows 10 operating system

More information about Windows 10 releases can be found online at:

<https://docs.microsoft.com/en-us/windows/release-health/release-information/>

According to official figures published by Microsoft, the public releases of the Windows 11 operating system are shown in Table 3.

Version	Build	Release
21H2	22000	October 2021
22H2	22621	September 2022
23H2	22631	October 2023
24H2	26100	October 2024
25H2	26200	September 2025

Table 3 Public releases of the Windows 11 operating system

More information about Windows 11 releases can be found online at:

<https://learn.microsoft.com/en-us/windows/release-health/windows11-release-information>

Mobile Devices

Windows Phone is a mobile operating system (smartphones) developed by Microsoft, succeeding the Windows Mobile platform. Windows Phone offered a completely new vision and functionality, in step with modern lifestyles and constant Internet connectivity, combined with powerful hardware that provided an optimal multimedia experience.



Fig. 1.6. Windows Phone 7 released in 2010

The first Windows Phone 7 was announced on October 21, 2010, localized into 25 languages, available in 35 countries, and its apps were published in the Windows Phone Store. It was based on Windows Embedded Compact 7 (a version of Windows Embedded CE). Support ended on October 14, 2014.



Fig. 1.7. Windows Phone 8 released in 2012

The second generation, Windows Phone 8, was released on October 29, 2012, and Windows Phone 8.1 on April 2, 2014. Windows CE was replaced by the Windows NT kernel used in Windows 8. Apps for the platform were published in the Windows Phone Store. Support for Windows Phone 8 ended on January 12, 2016, and for Windows Phone 8.1 on July 11, 2017.



Fig. 1.8. Windows 10 Mobile released in 2015

On November 20, 2015, the Windows 10 Mobile operating system was introduced. It supported the Universal Windows Platform architecture. Apps were published and downloaded from the Windows Store. Support ended on June 11, 2019.

Universal Platform

On September 13, 2011, during the Build conference, the Windows Store was introduced for distributing applications for PCs running Windows 8.

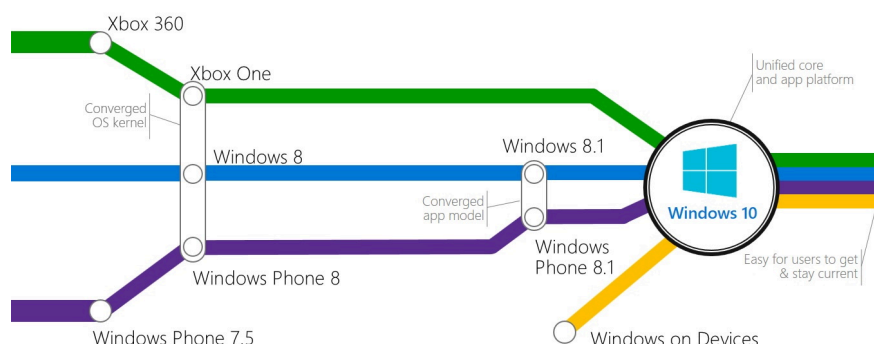


Fig. 1.9. History of Microsoft's application store evolution

When the Windows Phone platform was introduced in 2012, the Windows Phone Marketplace was renamed to Windows Phone Store, which provided distribution for apps targeting Windows Phone mobile devices.

With the release of Windows 10, the Windows Phone Store and Windows Store were unified. The operating system moved to a single architecture called the Windows Universal Platform. Today the company's application store is called the Microsoft Store.



Fig. 1.10. Architecture of the Universal Windows Platform for applications

This tutorial introduces the reader to the exciting world of Microsoft technologies for developing universal applications for the Windows platform.

Installation

A free version of the integrated development environment called **Microsoft Visual Studio Community Edition** can be downloaded from the company's official website at: <https://visualstudio.microsoft.com/>.

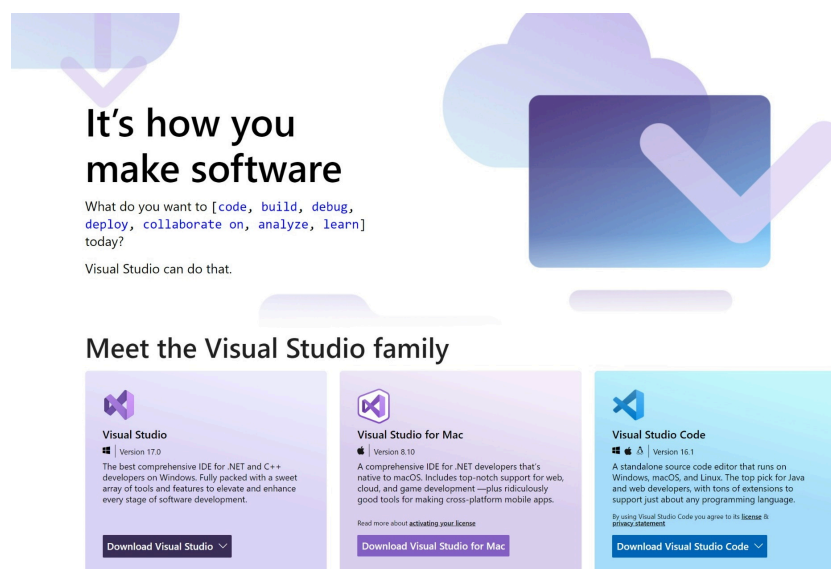


Fig. 1.11. Microsoft Visual Studio Website

This chapter uses **Microsoft Visual Studio Community Edition 2022**.

A default (standard) installation requires a modest **1.39 GB** of free disk space, but that includes only the IDE editor and basic collaboration tools.

For developing Universal Windows applications you should enable the **Universal Windows Platform development** workload (Fig. 1.12), and for developing cross-platform mobile applications enable **Mobile development with .NET** (Fig. 1.13)

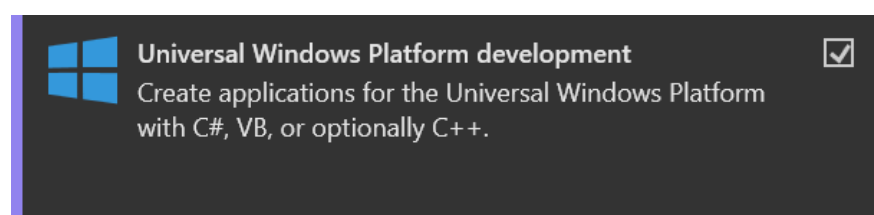


Fig. 1.12. Visual Studio Installer: Universal Windows Platform development

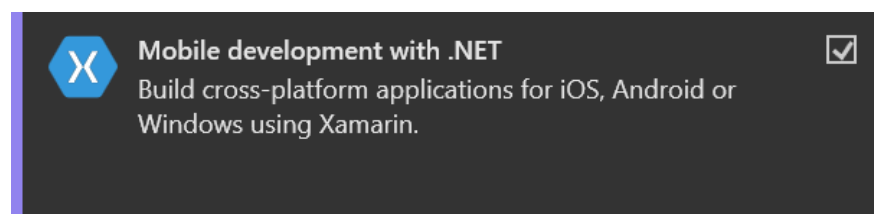


Fig. 1.13. Visual Studio Installer: Mobile development with .NET

Additional developer tools increase the required disk space for installing Visual Studio. Approximately **20.87 GB** of free disk space is required (*Fig. 1.14*).

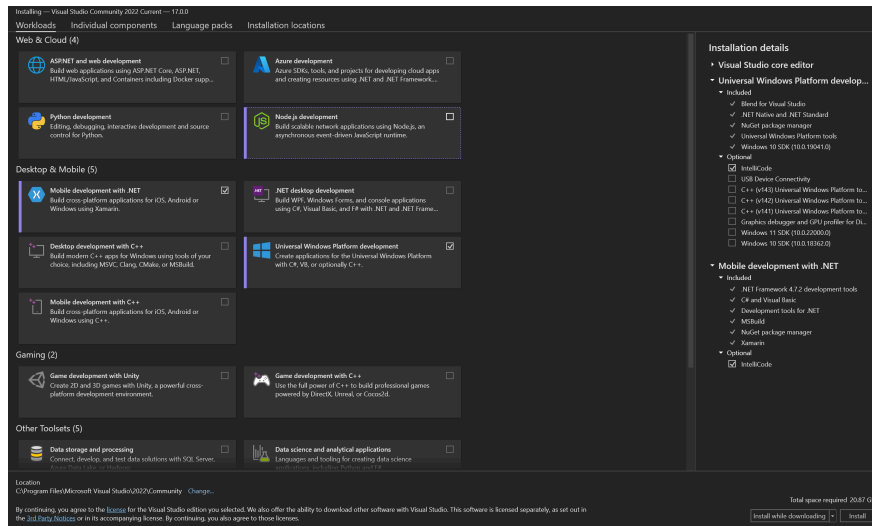


Fig. 1.14. Visual Studio Setup

Continue and wait for the installation to complete. The procedure may take a significant amount of time.

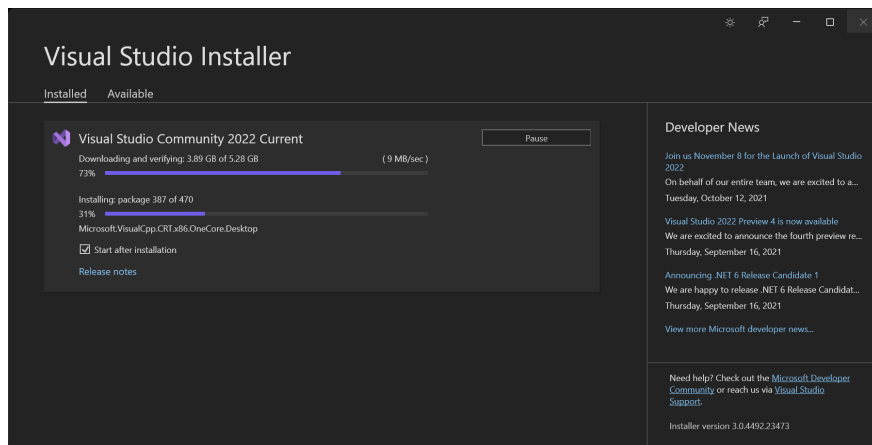


Fig. 1.15. Visual Studio Installer

Enabling Developer Mode

To develop Universal Windows Platform applications, you need to enable Developer Mode in the operating system, as shown in *Fig. 1.16 (a)* and *Fig. 1.16 (b)*.

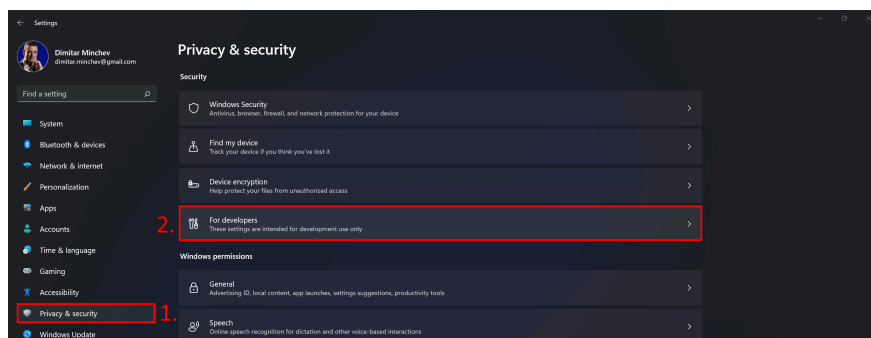


Fig. 1.16 (a). Enabling Developer Mode

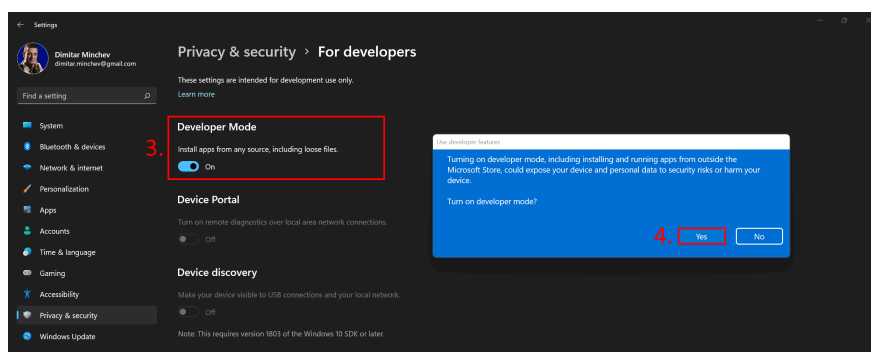


Fig. 1.16 (b). Enabling Developer Mode

Note

The illustrations show how to enable Developer Mode on Microsoft Windows 11.

Registering as an App Developer

To publish applications for the Windows platform you must register as a developer in the developer portal called the **Microsoft Partner Center**. The portal allows publishing to the application store known as the **Microsoft Store**.

Registration is available online at: <https://developer.microsoft.com/en-us/microsoft-store/register/>.

Developer registration requires a one-time fee in US dollars, which differs for individual and corporate accounts.

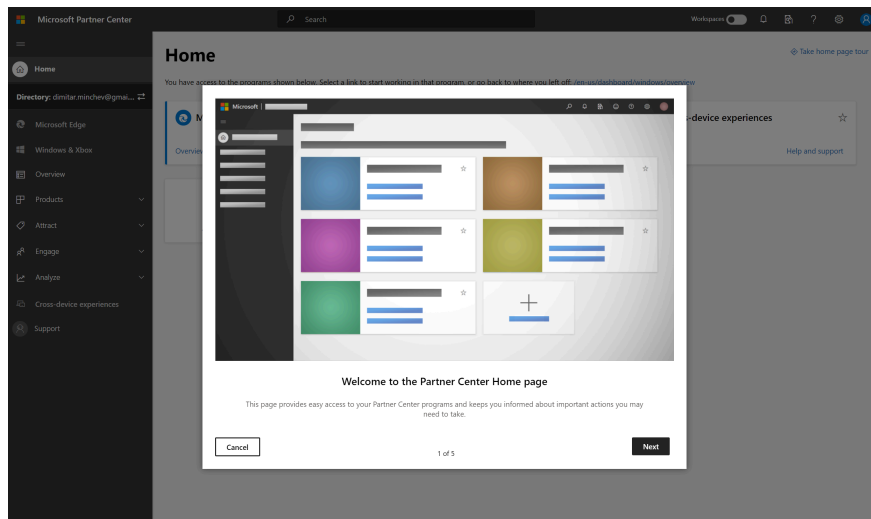


Fig. 1.17. Microsoft Partner Center

How to check the operating system version?

To check the current version of Windows, use the keyboard shortcut **Windows Key + R** (Fig. 1.18). In the Run dialog that appears, type the command **winver** (Fig. 1.19) and press **Enter**. The About Windows dialog (Fig. 1.20) will display the current operating system version.



Fig. 1.18. Keyboard shortcut Windows Key + R

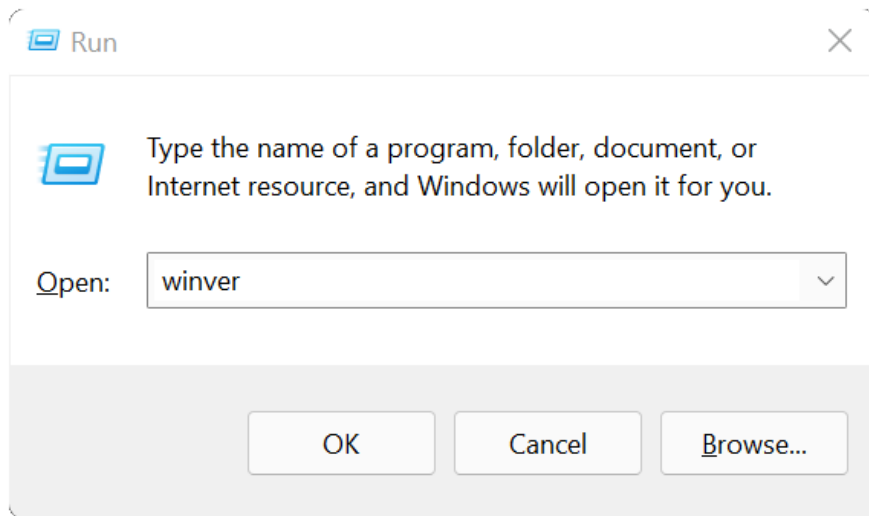


Fig. 1.19. RUN dialog



Fig. 1.20. Windows operating system version

Note

If you want to change the registered owner or organization name, open the **Registry Editor** and navigate to:

```
Computer\HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion
```

Modify the values for the fields:

- RegisteredOwner
- RegisteredOrganization

Applications

This section presents a number of applications demonstrating development of **Universal Windows Platform (UWP)** apps.

Development of each application in Microsoft Visual Studio starts by creating a project. A new project is created from the menu via: **File > New > Project**. You can also use the keyboard shortcut **Ctrl + Shift + N** to create a new project.

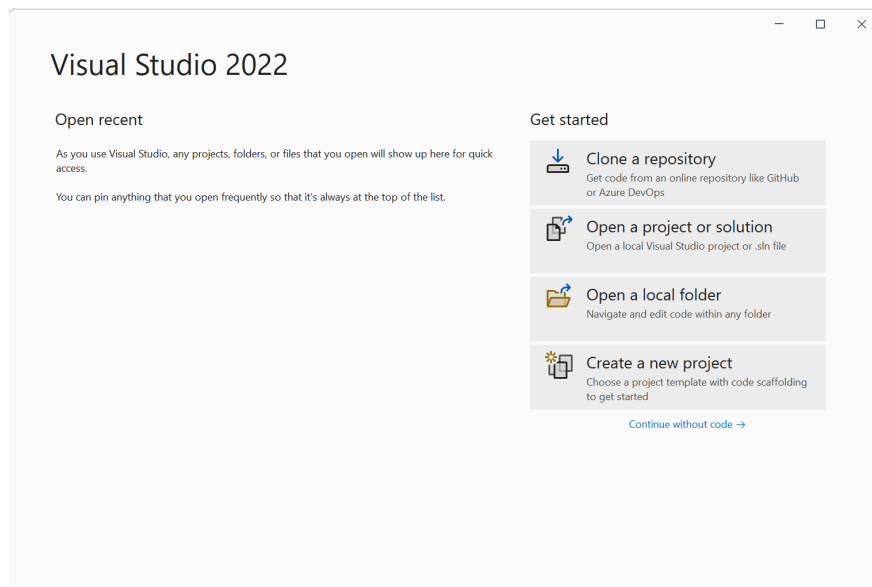


Fig. 1.21 (a). Creating a new project in Visual Studio

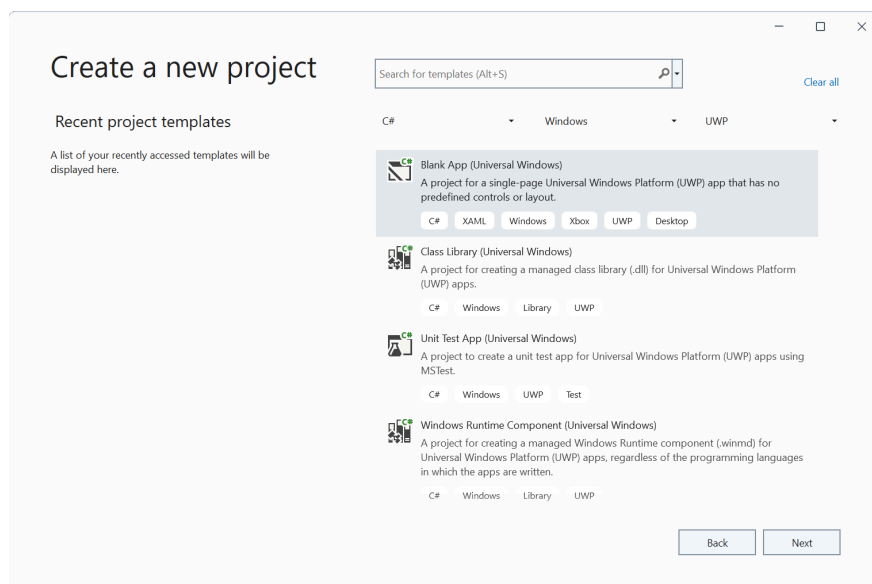


Fig. 1.21 (b). Creating a new project in Visual Studio

The image shows a window titled "Configure your new project" with standard Windows window controls (minimize, maximize, close) in the top right corner. Below the title bar, there are tabs for "Blank App (Universal Windows)", "C#", "XAML", "Windows", "Xbox", "UWP", and "Desktop". The "Blank App (Universal Windows)" tab is selected. The form contains three main sections: "Project name" with a text box containing "App1"; "Location" with a dropdown menu showing "C:\Users\mitko\Desktop\" and a browse button "..."; and "Solution name" with a text box containing "App1" and a help icon. Below these is a checkbox labeled "Place solution and project in the same directory" which is currently unchecked. At the bottom right, there are two buttons: "Back" and "Create".

Configure your new project

Blank App (Universal Windows) C# XAML Windows Xbox UWP Desktop

Project name

App1

Location

C:\Users\mitko\Desktop\

Solution name ⓘ

App1

☐ Place solution and project in the same directory

Back Create

Fig. 1.21 (c). Creating a new project in Visual Studio

Suma 1.0

Using Visual Studio and the C# programming language we will build a Universal Windows Platform application that computes the sum of two numbers.

Start

Launch Visual Studio. Create a new project from the menu via: **File > New > Project**, or use the shortcut **Ctrl + Shift + N**. In the dialog that appears choose: **Visual C# > Windows Universal > Blank App (Universal Windows)**. Name the project **Suma 1.0**.

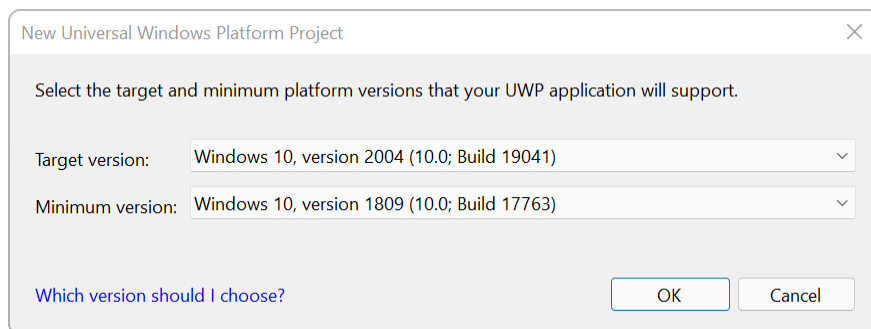


Fig. 1.22. Selecting the Windows version

From Solution Explorer open **MainPage.xaml** and **MainPage.xaml.cs**. If you don't see Solution Explorer, open it from **View > Solution Explorer** or use the shortcut **Ctrl + W, S**.

MainPage.xaml

The **MainPage.xaml** file contains the user interface markup written in XAML. Copy (Ctrl+C) and paste (Ctrl+V) the following fragment into your application.

```

<!-- User Interface (UI): Suma 1.0 -->
<StackPanel Padding="50" Background="Orange">

    <!-- Title -->
    <TextBlock Text="Suma 1.0" FontSize="42" />

    <!-- A -->
    <TextBlock Text="A=" FontSize="24" />
    <TextBox x:Name="boxA" FontSize="24" />

    <!-- B -->
    <TextBlock Text="B=" FontSize="24" />
    <TextBox x:Name="boxB" FontSize="24" />

    <!-- A+B -->
    <TextBlock Text="A+B=" FontSize="24" />
    <TextBox x:Name="boxAB" FontSize="24" />

    <!-- Button -->
    <Button Content="Sum" FontSize="24"
           Margin="0 20 0 0" Padding="20 10 20 10"
           Click="OnButtonClicked" />

</StackPanel>

```

Design view (XAML) in Visual Studio while developing the application:

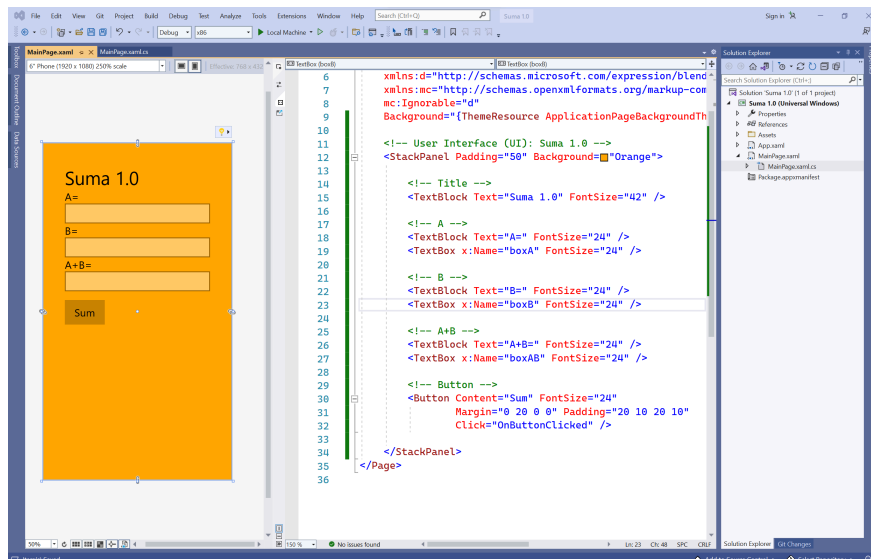


Fig. 1.23. UI design view

MainPage.xaml.cs

The **MainPage.xaml.cs** file contains the business logic written in C#. Copy (Ctrl+C) and paste (Ctrl+V) the following fragment into your application.

```
// Business Logic (BL): Suma 1.0

public sealed partial class MainPage : Page
{
    // Constructor
    public MainPage()
    {
        this.InitializeComponent();
    }

    // Button Click Event Handler
    private void OnButtonClicked(object sender, RoutedEventArgs e)
    {
        var A = int.Parse(this.boxA.Text);
        var B = int.Parse(this.boxB.Text);
        var C = A + B;
        this.boxAB.Text = C.ToString();
    }
}
```

Business logic (C#) view in Visual Studio while developing the application:

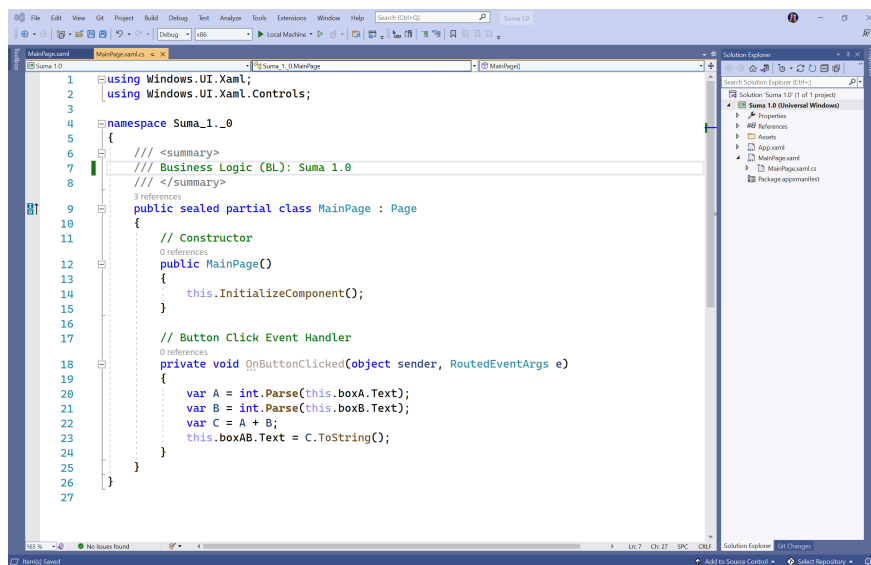


Fig. 1.24. Business logic view

Demo

Start the app via **Debug > Start Debugging** or press **F5**.



Fig. 1.25. Universal app that computes the sum of two numbers

Fibonacci 1.0

Using Visual Studio and C# we will build a Universal Windows Platform application that generates numbers from the Fibonacci sequence.

Info

In mathematics, the Fibonacci numbers form a sequence defined recursively as follows: start with 0 and 1, and each subsequent term is the sum of the previous two. The first Fibonacci numbers are: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, ...

- Source: [Wikipedia](#)

Start

Launch **Visual Studio**. Create a new project via **File > New > Project** or use the shortcut **Ctrl + Shift + N**. In the dialog choose: **Visual C# > Windows Universal > Blank App (Universal Windows)**. Name the project **Fibonacci 1.0**. From Solution Explorer open **MainPage.xaml** and **MainPage.xaml.cs**. If you don't see Solution Explorer, open it from **View > Solution Explorer** or use **Ctrl + W, S**.

MainPage.xaml

The **MainPage.xaml** file contains the user interface markup written in XAML. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application.

```
<!-- User Interface (UI): Fibonacci 1.0 -->
<StackPanel Background="Pink" Padding="50">

    <!-- Title -->
    <TextBlock Text="Fibonacci 1.0" FontSize="40" />

    <!-- Limit -->
    <TextBlock Text="Limit" Margin="0 10 0 10" FontSize="20" />
    <TextBox Name="boxLimit" FontSize="20" Text="1000" />
    <Button Content="Generate" Margin="0 10 0 10" Padding="20 10 20 10" FontSize="20" />

    <!-- Numbers -->
    <ListBox Name="boxNumbers" Height="400" FontSize="20" />

</StackPanel>
```

Design view (XAML) in Visual Studio while developing the application:

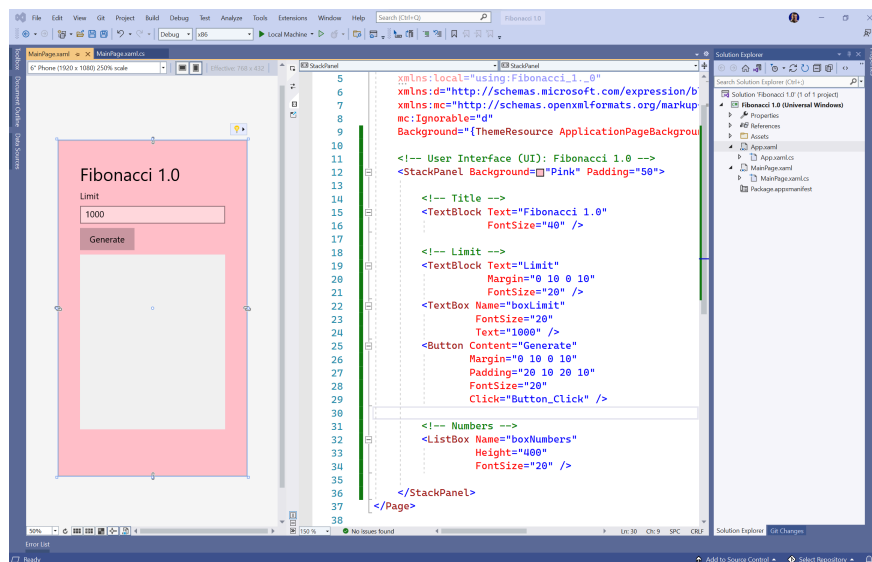


Fig. 1.26. UI design view

MainPage.xaml.cs

The **MainPage.xaml.cs** file contains the business logic written in C#. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application.

```
// Business Logic (BL): Fibonacci 1.0
public sealed partial class MainPage : Page
{
    // Collection
    private ObservableCollection<int> numbers = new ObservableCollection<int>();

    // Constructor
    public MainPage()
    {
        this.InitializeComponent();
        boxNumbers.ItemsSource = numbers;
    }

    // Button Click Event Handler
    private void Button_Click(object sender, RoutedEventArgs e)
    {
        numbers.Add(1);
        numbers.Add(1);
        int limit = int.Parse(boxLimit.Text);
        int a = 1, b = 1, c = a + b;
        while (c < limit)
        {
            numbers.Add(c);
            a = b;
            b = c;
            c = a + b;
        }
    }
}
```

Business logic (C#) view in Visual Studio while developing the application:

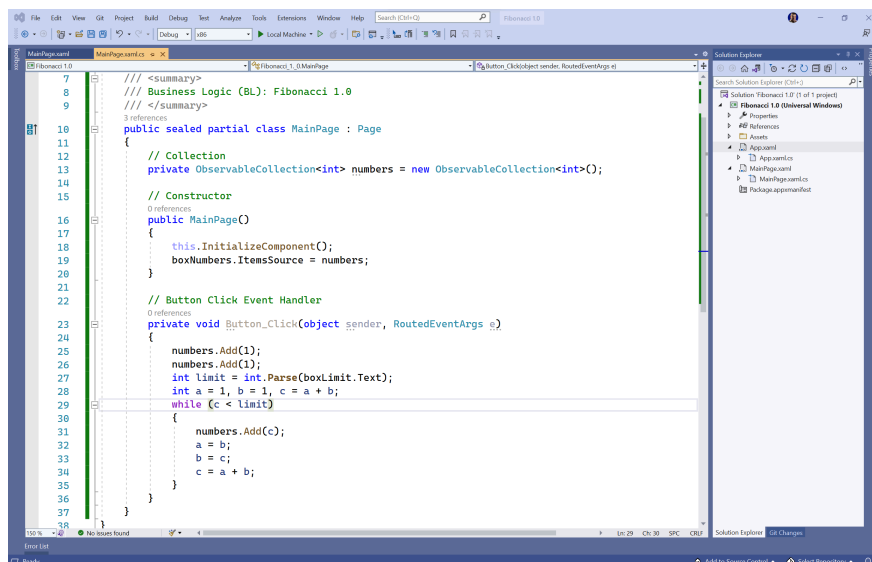


Fig. 1.27. Business logic view

Demo

Start the app via **Debug > Start Debugging** or press **F5**.



Fig. 1.28. Universal app that generates numbers from the Fibonacci sequence.

Primes 1.0

Using Visual Studio and C# we will build a Universal Windows Platform application that generates a sequence of prime numbers.

Info

A prime number is a natural number greater than 1 that has no positive divisors other than 1 and itself.

- Source: [Wikipedia](#)

Start

1. Launch **Visual Studio**.
2. Create a new project: **Visual C# > Windows Universal > Blank App (Universal Windows)**.
3. Name the project **Primes 1.0**.

MainPage.xaml

The **MainPage.xaml** file contains the user interface markup written in XAML. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application.

```
<!-- User Interface (UI): Primes 1.0 -->
<StackPanel Background="Yellow" Padding="50">

    <!-- Title -->
    <TextBlock Text="Primes 1.0" FontSize="40" />

    <!-- Limit -->
    <TextBlock Text="Limit" FontSize="20" />
    <TextBox Name="boxLimit" Text="1000" FontSize="20" />
    <Button Content="Generate" Margin="0 10 0 10" Padding="20 10 20 10" FontSize="20" />

    <!-- Numbers -->
    <ListBox Name="boxNumbers" Height="400" FontSize="20" />

</StackPanel>
```

Design view (XAML) in Visual Studio while developing the application:

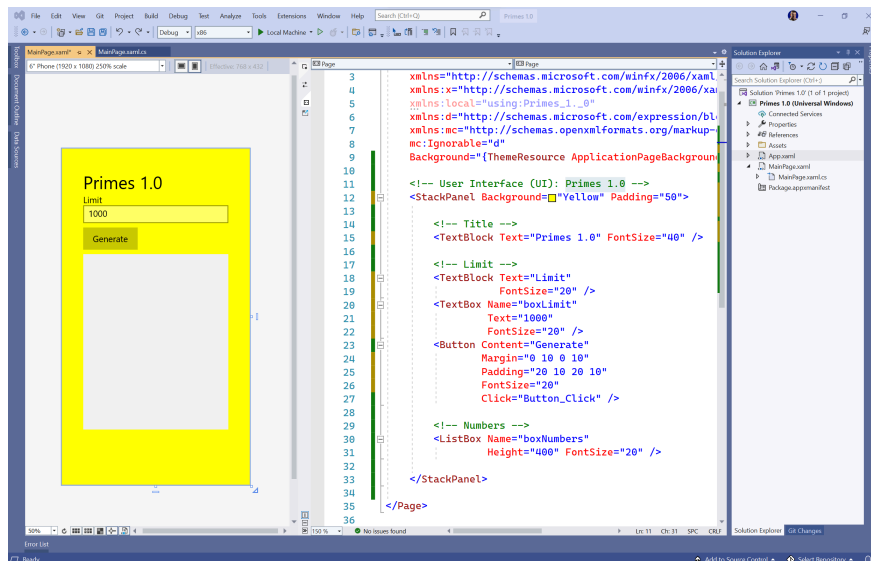


Fig. 1.29. UI design view

MainPage.xaml.cs

The **MainPage.xaml.cs** file contains the business logic written in C#. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application.

```
// Business Logic (BL): Primes 1.0

public sealed partial class MainPage : Page
{
    // Constructor
    public MainPage()
    {
        this.InitializeComponent();
    }

    // Button Event Handler
    private void Button_Click(object sender, RoutedEventArgs e)
    {
        int limit = int.Parse(boxLimit.Text);
        for (int k = 2; k < limit; k++)
        {
            bool prime = true;
            for (int j = 2; j < k; j++) if (k % j == 0) prime = false;
            if (prime) boxNumbers.Items.Add(k);
        }
    }
}
```

Business logic (C#) view in Visual Studio while developing the application:

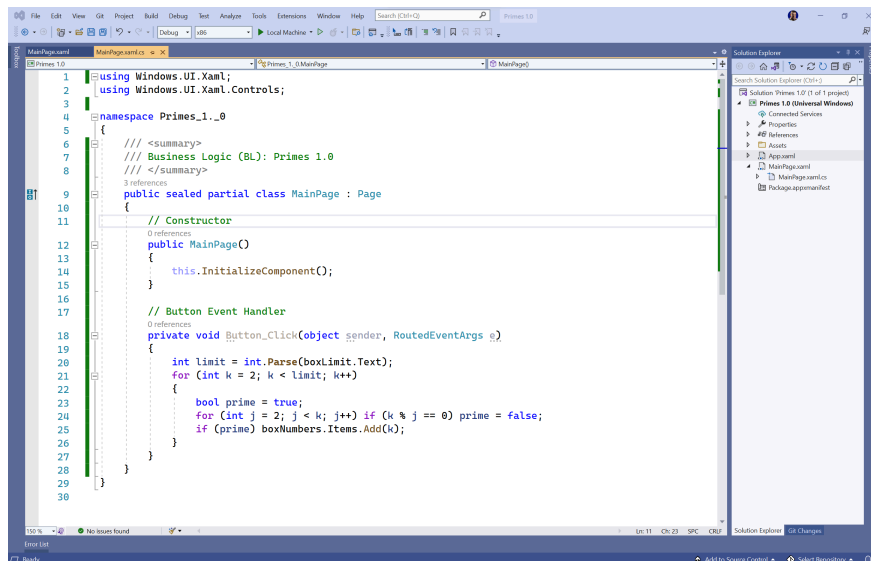


Fig. 1.30. Business logic view

Demo

Start the app via **Debug > Start Debugging** or press **F5**.

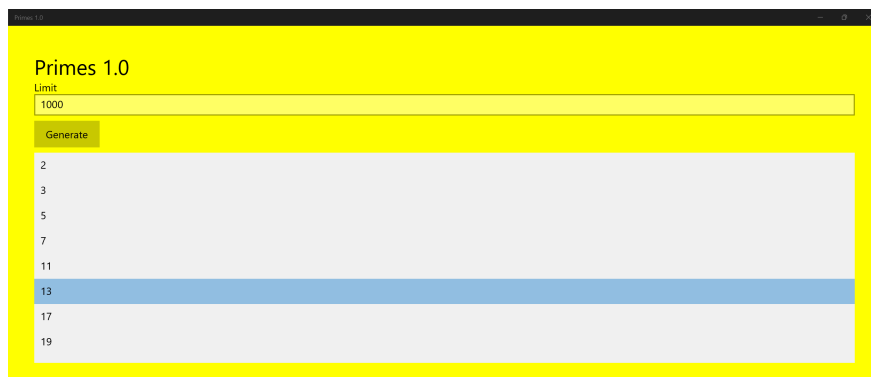


Fig. 1.31. Universal app that generates a sequence of prime numbers.

Phone Book 1.0

Using Visual Studio and C# we will build a Universal Windows Platform phone book application that displays a list of contacts. The UI (XAML) and business logic (C#) code snippets are shown below.

Contact.cs

```
public class Contact
{
    public Uri picture { get; set; }
    public string name { get; set; }
    public string phone { get; set; }

    public Contact(Uri _picture, string _name, string _phone)
    {
        this.picture = _picture;
        this.name = _name;
        this.phone = _phone;
    }
}
```

App.xaml.cs

```
/// <summary>
/// Provides application-specific behavior to supplement the default Application class
/// </summary>
sealed partial class App : Application
{
    // Contacts
    public static ObservableCollection<Contact> contacts = new ObservableCollection<Co
    ...
```

AddPage.xaml

```
<!-- User Interface (UI): Phone Book 1.0 -->
<StackPanel Background="Cyan" Padding="40">

    <!-- Title -->
    <TextBlock Text="Add Contact" FontSize="40" />

    <!-- Name -->
    <TextBlock Text="Name" FontSize="20" />
    <TextBox Name="boxName" FontSize="20" />

    <!-- Phone -->
    <TextBlock Text="Phone" FontSize="20" />
    <TextBox Name="boxPhone" FontSize="20" />

    <!-- Picture -->
    <TextBlock Text="Picture" FontSize="20" />
    <TextBox Name="boxPicture" FontSize="20" />

    <!-- Button -->
    <Button Content="Add Contact" FontSize="20" Padding="20 10 20 10" Margin="0 20

</StackPanel>
```

AddPage.xaml.cs

```
// Business Logic (BL): Phone Book 1.0
public sealed partial class AddPage : Page
{
    // Constructor
    public AddPage()
    {
        this.InitializeComponent();
    }

    // Button Click Event Handler
    private void Button_Click(object sender, RoutedEventArgs e)
    {
        var picture = new Uri(boxPicture.Text);
        var contact = new Contact(picture, boxName.Text, boxPhone.Text);
        this.Frame.Navigate(typeof(MainPage), contact);
    }
}
```

MainPage.xaml

```
<!-- User Interface (UI): Phone Book 1.0 -->
<StackPanel Background="Cyan" Padding="40">

    <!-- Title -->
    <TextBlock Text="Phone Book 1.0" FontSize="40" />

    <!-- Contacts -->
    <ListBox Name="boxContacts" Height="400">
        <ListBox.ItemTemplate>
            <DataTemplate>
                <StackPanel Orientation="Horizontal">
                    <Image Source="{Binding picture}" Width="100" Height="100" />
                    <StackPanel>
                        <TextBlock Text="{Binding name}" FontSize="32" />
                        <TextBlock Text="{Binding phone}" FontSize="32" />
                    </StackPanel>
                </StackPanel>
            </DataTemplate>
        </ListBox.ItemTemplate>
    </ListBox>

    <!-- Add -->
    <Button Content="Add Contact" FontSize="20" Padding="20 10 20 10" Margin="0 20 0 20" />

</StackPanel>
```

MainPage.xaml.cs

```
// Business Logic (BL): Phone Book 1.0
public sealed partial class MainPage : Page
{
    // Constructor
    public MainPage()
    {
        this.InitializeComponent();
        boxContacts.ItemsSource = App.contacts;
    }

    // Button Click Event Handler
    private void Button_Click(object sender, RoutedEventArgs e)
    {
        this.Frame.Navigate(typeof(AddPage));
    }

    // Navigation Event Handler
    protected override void OnNavigatedTo(NavigationEventArgs e)
    {
        if (e.Parameter is Contact)
        {
            App.contacts.Add(e.Parameter as Contact);
        }
    }
}
```

Demo

Design view in Visual Studio while developing the application:

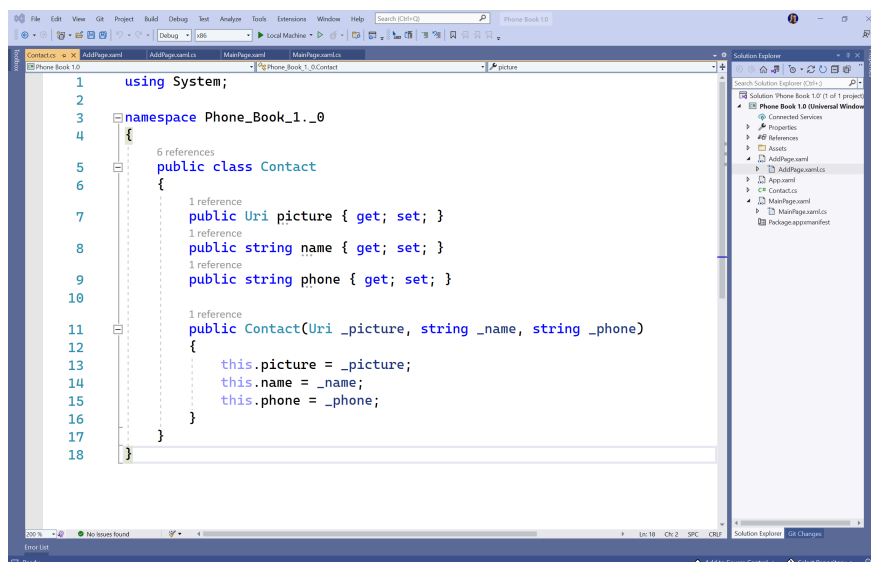


Fig. 1.32. Visual Studio view during app development

Start the app via **Debug > Start Debugging** or press **F5**.

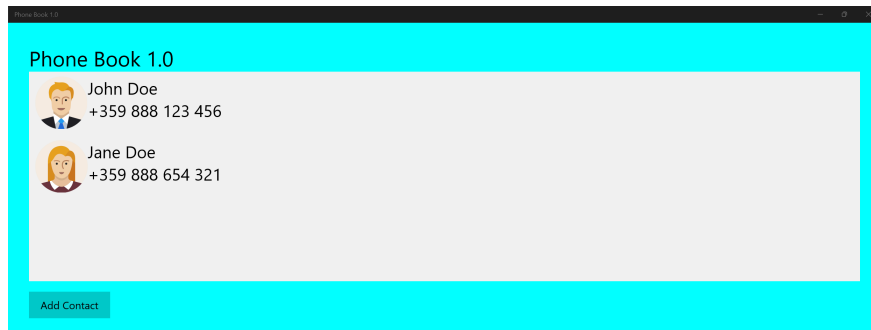


Fig. 1.33. Universal phone book app showing a list of contacts

Profile images used

1. Male: <https://icons-for-free.com/iconfiles/png/512/business+costume+male+man+office+user+icon-1320196264882354682.png>
2. Female: <https://icons-for-free.com/iconfiles/png/512/female+person+user+woman+young+icon-1320196266256009072.png>

Clicker Mania 1.1

Using Visual Studio and C# we will build a Universal Windows Platform application that counts user clicks.

Start

1. Launch **Visual Studio**.
2. Create a new project: **Visual C# > Windows Universal > Blank App (Universal Windows)**.
3. Name the project **Clicker Mania 1.1**.

MainPage.xaml

The **MainPage.xaml** file contains the UI markup written in XAML. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application.

```
<!-- User Interface (UI): Clicker Mania 1.1 -->
<StackPanel Background="Orange" Padding="50">

    <!-- Title -->
    <TextBlock FontSize="40"
               HorizontalAlignment="Center"
               Text="Clicker Mania 1.1" />

    <!-- Clicks -->
    <TextBlock Name="Clicks"
               HorizontalAlignment="Center"
               Text="0" FontSize="100" Padding="50"
               FontWeight="Black" />

    <!-- Button -->
    <Button Content="Click"
            HorizontalAlignment="Center"
            FontSize="40"
            Padding="40 20 40 20"
            Click="Button_Click" />

</StackPanel>
```

Design view (XAML) in Visual Studio while developing the application:

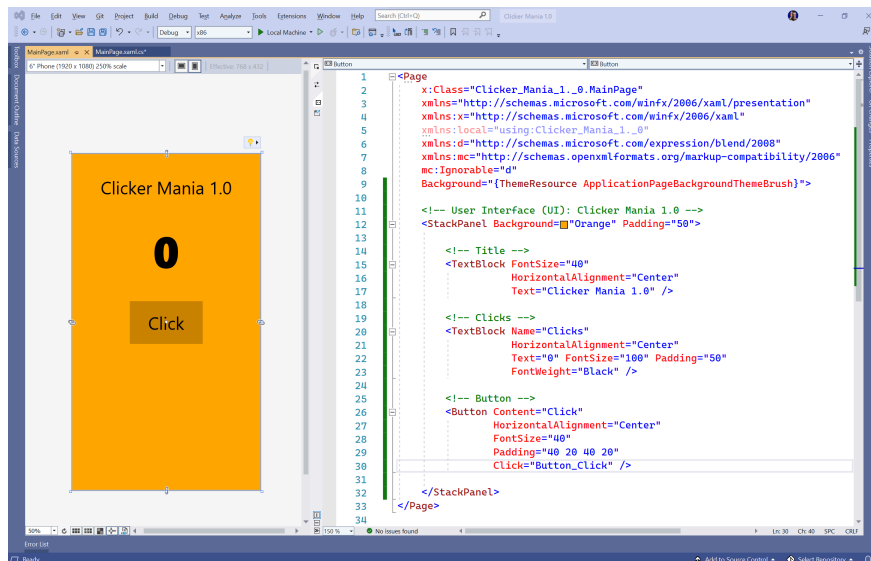


Fig. 1.34. UI design view

MainPage.xaml.cs

The **MainPage.xaml.cs** file contains the business logic written in C#. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application.

```

// Business Logic (BL): Clicker Mania 1.1
public sealed partial class MainPage : Page
{
    // Counter
    private int counter = 0;

    // Constructor
    public MainPage()
    {
        this.InitializeComponent();
        Read();
    }

    // Button Click Event Handler
    private void Button_Click(object sender, RoutedEventArgs e)
    {
        counter = counter + 1;
        Clicks.Text = counter.ToString();
        Save(Clicks.Text);
    }

    // Save to File
    private async void Save(string content)
    {
        try
        {
            StorageFolder storage = ApplicationData.Current.LocalFolder;
            byte[] bytes = Encoding.UTF8.GetBytes(content.ToCharArray());
            var file = await storage.CreateFileAsync("clicker.txt", CreationCollisionOptions.ReplaceExisting)
            using (var stream = await file.OpenStreamForWriteAsync())
            {
                stream.Write(bytes, 0, bytes.Length);
            }
        }
        catch
        {
            // On Error
        }
    }

    // Read from File
    private async void Read()
    {
        try
        {
            StorageFolder storage = ApplicationData.Current.LocalFolder;
            StorageFile file = await storage.GetFileAsync("clicker.txt");

```

```

        if (file == null)
        {
            file = await storage.CreateFileAsync("clicker.txt");
        }
        else
        {
            Stream stream = await file.OpenStreamForReadAsync();
            StreamReader reader = new StreamReader(stream);
            Clicks.Text = reader.ReadToEnd();

            if (Clicks.Text == "")
            {
                Clicks.Text = "0";
                counter = 0;
            }
            else counter = int.Parse(Clicks.Text);
        }
    }
    catch
    {
        // On Error
    }
}
}

```

Business logic (C#) view in Visual Studio while developing the application:

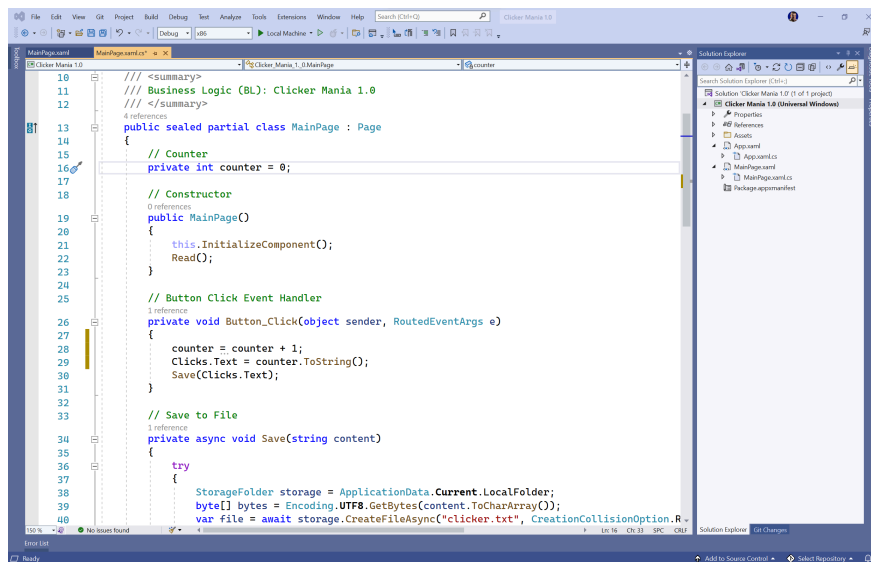


Fig. 1.35. Business logic view

Demo

Start the app via **Debug > Start Debugging** or press **F5**.

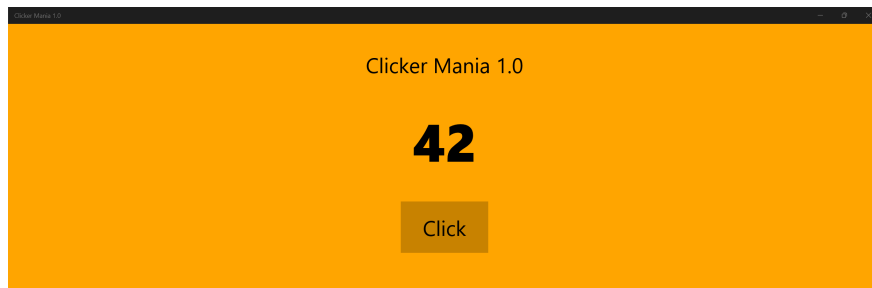


Fig. 1.36. Universal app that counts user clicks.

Clicker Mania 1.2

Using Visual Studio and C# we will build a Universal Windows Platform application that counts user clicks over a given period.

Start

1. Launch **Visual Studio**.
2. Create a new project: **Visual C# > Windows Universal > Blank App (Universal Windows)**.
3. Name the project **Clicker Mania 1.2**.

MainPage.xaml

The **MainPage.xaml** file contains the UI markup written in XAML. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application.

```
<!-- User Interface (UI): Clicker Mania 1.2 -->
<StackPanel Background="Lime" Padding="30">

    <!-- Title -->
    <TextBlock FontSize="30" Padding="10" HorizontalAlignment="Center" Text="Clicker Mania 1.2" />

    <!-- Timer -->
    <TextBlock Text="Timer" FontSize="23" HorizontalAlignment="Center" />
    <TextBlock Name="Timer" Foreground="White" Text="0" FontSize="60" HorizontalAlignment="Center" />

    <!-- Clicks -->
    <TextBlock Text="Clicks" FontSize="23" HorizontalAlignment="Center" />
    <TextBlock Name="Clicks" Foreground="White" Text="0" FontSize="60" HorizontalAlignment="Center" />

    <!-- Clicks Per Minute -->
    <TextBlock Text="Clicks Per Minute" FontSize="23" HorizontalAlignment="Center" />
    <TextBlock Name="CPM" Foreground="White" Text="0" FontSize="60" HorizontalAlignment="Center" />

    <!-- Button -->
    <Button Content="Click" Padding="50 10 50 10" HorizontalAlignment="Center" FontSize="23" />

</StackPanel>
```

Design view (XAML) in Visual Studio while developing the application:

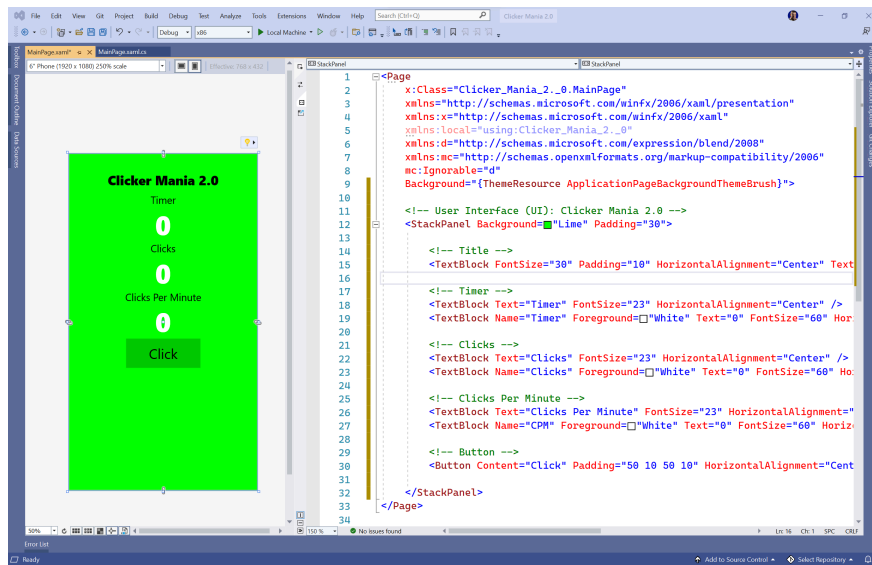


Fig. 1.37. UI design view

MainPage.xaml.cs

The **MainPage.xaml.cs** file contains the business logic written in C#. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application.


```

// Business Logic (BL): Clicker Mania 1.2
public sealed partial class MainPage : Page
{
    // Dispatcher Timer
    private DispatcherTimer timer = new DispatcherTimer();

    // Constructor
    public MainPage()
    {
        this.InitializeComponent();
        // Timer
        timer.Interval = new TimeSpan(0, 0, 1);
        timer.Tick += Timer_Tick;
        timer.Start();
    }

    // Timer Tick Event Handler
    private void Timer_Tick(object sender, object e)
    {
        int T = int.Parse(Timer.Text);
        Timer.Text = (++T).ToString();
        // Clicks Per Minute
        CPM.Text = (float.Parse(Clicks.Text) / float.Parse(Timer.Text) * 60).ToStr
    }

    // Button Click Event Handler
    private void Button_Click(object sender, RoutedEventArgs e)
    {
        int N = int.Parse(Clicks.Text);
        Clicks.Text = (++N).ToString();
    }
}

```

Business logic (C#) view in Visual Studio while developing the application:

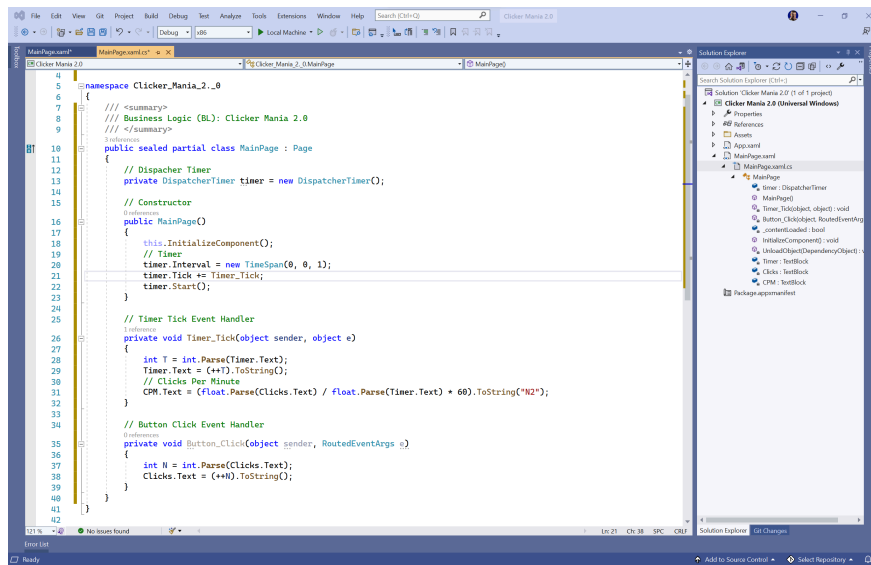


Fig. 1.38. Business logic view

Demo

Start the app via **Debug > Start Debugging** or press **F5**.

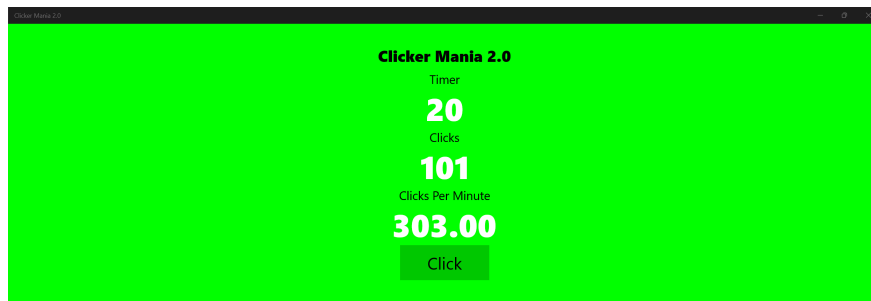


Fig. 1.39 Universal app that counts user clicks over a given period

HTML Downloader 1.0

Using the Visual Studio integrated development environment and the C# programming language, we will develop a universal application to download the HTML content of a web page.

Information

HTML is the standard markup language for describing and designing web pages. HTML is a standard on the Internet and its rules are defined by the World Wide Web Consortium (W3C).

- Source: [Wikipedia](#)

Start

1. Launch the Visual Studio integrated development environment.
2. Create a new project: **Visual C# > Windows Universal > Blank App (Universal Windows)**.
3. Name the project: **HTML Downloader 1.0**.

From the **Project > Manage NuGet Packages** menu, search for and install the **AngleSharp** package, as shown in the figure:

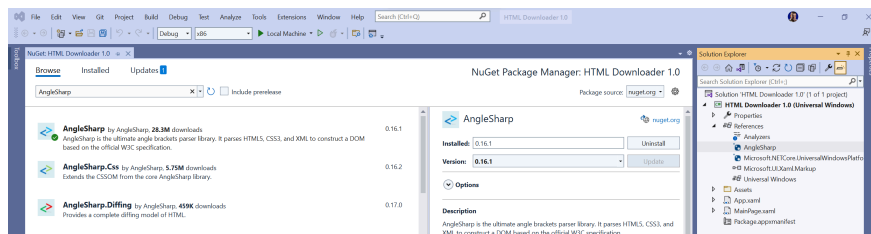


Fig. 40. Installing an additional package to the project

You can also install additional packages for the project alternatively from: **Tools > NuGet Package Manager > Package Manager Console**, by running the following command in the console:

```
PM> Install-Package AngleSharp -Version 0.16.1
```

MainPage.xaml

The **MainPage.xaml** file contains the source code for the application's user interface design and is written in XAML. Copy (Ctrl+C) and paste (Ctrl+V) the program fragment below into your application.

```

<!-- User Interface (UI): HTML Downloader 1.0 -->
<StackPanel Background="LightCoral" Padding="20">

    <!-- Title -->

    <TextBlock Text="HTML Downloader 1.0" FontSize="40" />

    <!-- URL -->

    <TextBlock Text="URL" FontSize="20" />
    <TextBox Name="URL" Text="http://www.minchev.eu" FontSize="20" />
    <Button Content="Download" Margin="0 10" Padding="20 10" FontSize="20" Click="

    <!-- HTML -->

    <TextBox Name="HTML" Height="400" TextWrapping="Wrap" IsReadOnly="True" FontSi

</StackPanel>

```

View of the user interface design (XAML) in Visual Studio while developing the application:

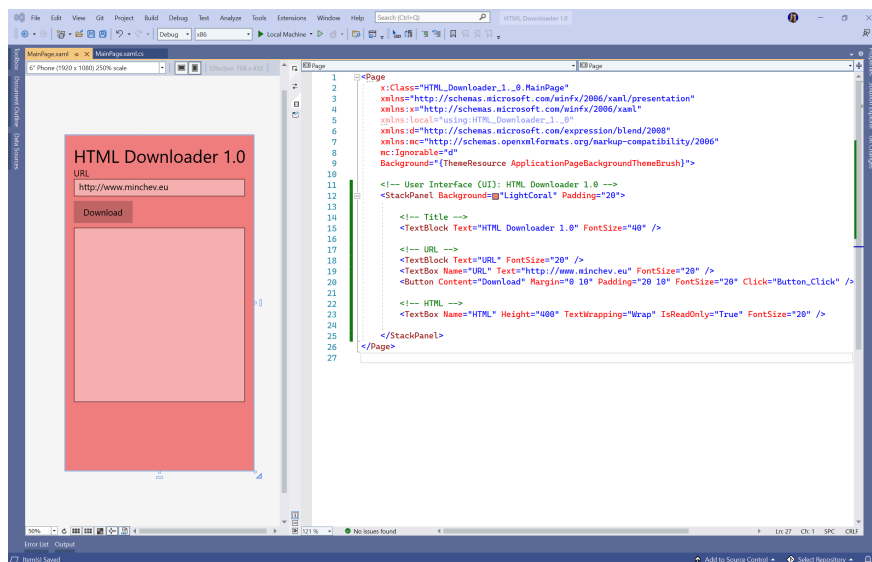


Fig. 1.41. View of the user interface design

MainPage.xaml.cs

The **MainPage.xaml.cs** file contains the source code for the application's business logic and is written in C#. Copy (Ctrl+C) and paste (Ctrl+V) the program fragment below into your application.

```
// Business Logic (BL): HTML Downloader 1.0
public sealed partial class MainPage : Page
{
    // Constructor
    public MainPage()
    {
        this.InitializeComponent();
    }

    // Button Click Event Handler
    private async void Button_Click(object sender, RoutedEventArgs e)
    {
        string html = await Download(new Uri(URL.Text));
        var temp = new HtmlParser().ParseDocument(html);
        string text = temp.Body.TextContent;
        HTML.Text = text;
    }

    // Download Handler
    private async Task<string> Download(Uri link)
    {
        HttpClient client = new HttpClient();
        return await client.GetStringAsync(link);
    }
}
```

Demo

View of the business logic (C#) in Visual Studio while developing the application:

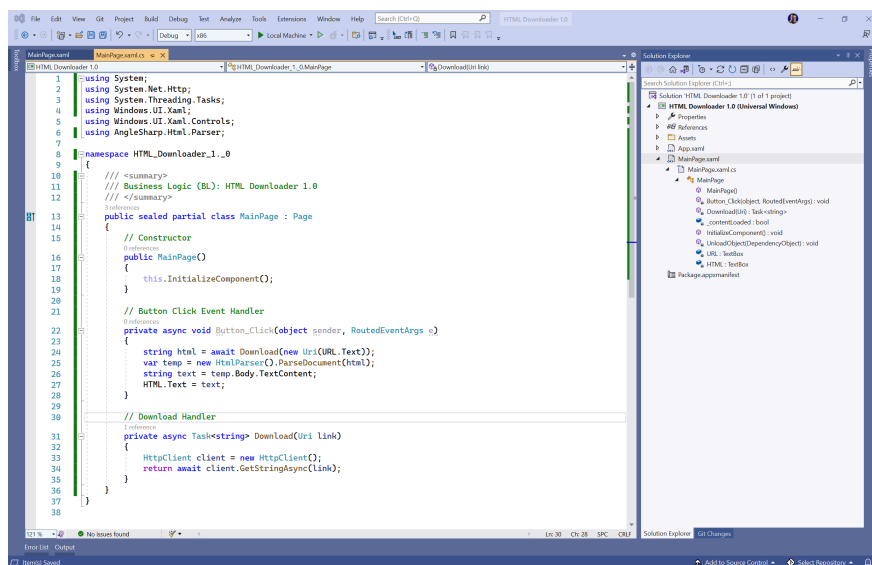


Fig. 1.42. View of the application's business logic

Run the application from the menu: **Debug > Start Debugging** or by pressing the **F5** key.



Fig. 1.43 Universal application for downloading the HTML content of a web page

RSS Reader 1.0

Using the Visual Studio integrated development environment and the C# programming language, we will develop a universal application for reading news feeds (RSS) from internet sources.

Information

RSS is a software mechanism for exchanging news between websites or between a site and a user. It represents a set of formats for delivering information from the World Wide Web.

- Source: [Wikipedia](#)

Start

1. Launch the Visual Studio integrated development environment.
2. Create a new project: **Visual C# > Windows Universal > Blank App (Universal Windows)**.
3. Name the project: **RSS Reader 1.0**.

Item.cs

Add a new class `Item.cs` which will be used to store data for each news item in the feed.

```
public class Item
{
    public string Title { get; set; }
    public string Link { get; set; }
    public string PublishedDate { get; set; }
}
```

MainPage.xaml

The **MainPage.xaml** file contains the source code for the application's user interface design and is written in XAML. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application.

```

<!-- User Interface (UI): RSS Reader 1.0 -->
<StackPanel Background="Lime" Padding="20">

    <!-- Title -->
    <TextBlock Text="RSS Reader 1.0" FontSize="40" />

    <!-- URI -->
    <TextBox Name="URI" Text="https://www.minchev.eu/feed/" FontSize="20" />
    <Button Content="Download" Margin="0 10" Padding="20 10" FontSize="20" Click="

    <!-- RSS -->
    <ScrollView>
        <ListView Name="RSS">
            <ListView.ItemTemplate>
                <DataTemplate>
                    <StackPanel>
                        <HyperlinkButton NavigateUri="{Binding Link}">
                            <HyperlinkButton.Content>
                                <TextBlock TextWrapping="Wrap" Text="{Binding Title}" />
                            </HyperlinkButton.Content>
                        </HyperlinkButton>
                        <TextBlock Text="{Binding PublishedDate}" />
                    </StackPanel>
                </DataTemplate>
            </ListView.ItemTemplate>
        </ListView>
    </ScrollView>
</StackPanel>

```

View of the user interface design (XAML) in Visual Studio while developing the application:

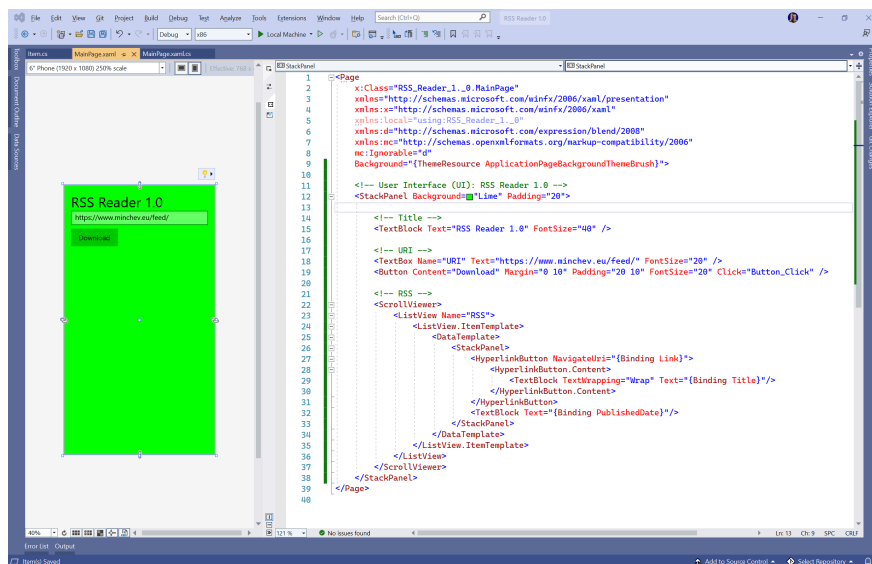


Fig. 1.44. View of the user interface design

MainPage.xaml.cs

The **MainPage.xaml.cs** file contains the source code for the application's business logic and is written in C#. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application.

```

// Business Logic (BL): RSS Reader 1.0
public sealed partial class MainPage : Page
{
    // View Model
    private ObservableCollection<Item> Items = new ObservableCollection<Item>();

    // Constructor
    public MainPage()
    {
        this.InitializeComponent();
        RSS.ItemsSource = Items;
    }

    // Button Click Event Handler
    private void Button_Click(object sender, RoutedEventArgs e)
    {
        Download();
    }

    // Download Handler
    private async void Download()
    {
        var uri = new Uri(URI.Text);
        var client = new SyndicationClient();
        var feed = await client.RetrieveFeedAsync(uri);
        if (feed != null)
        {
            foreach (var node in feed.Items)
            {
                Items.Add(new Item
                {
                    Title = node.Title.Text,
                    Link = node.Id,
                    PublishedDate = node.PublishedDate.ToString()
                });
            }
        }
    }
}

```

Demo

View of the business logic (C#) in Visual Studio while developing the application:

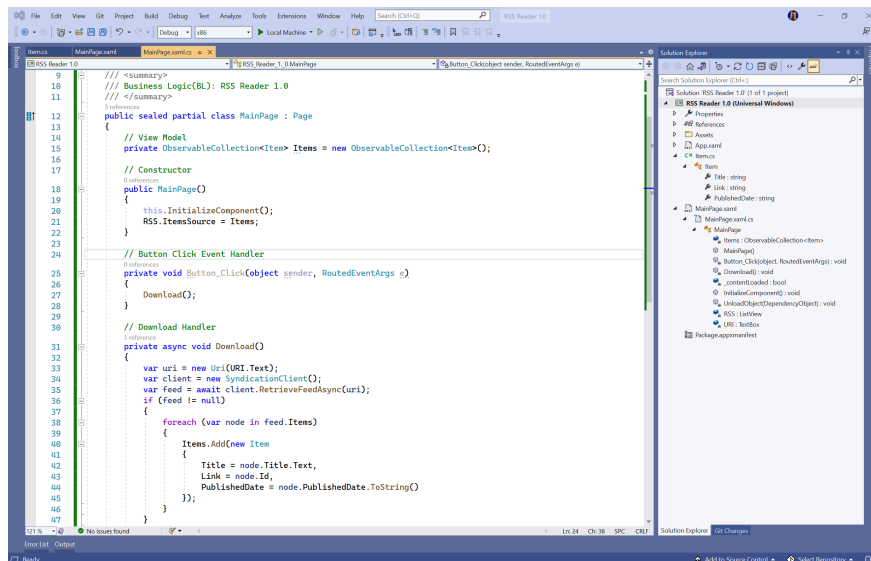


Fig. 1.45. View of the application's business logic

Run the application from the menu: **Debug > Start Debugging** or by pressing the **F5** key.



Fig. 1.46 Universal application for reading internet news feeds (RSS sources)

JSON Reader 1.0

Using the Visual Studio integrated development environment and the C# programming language, we will develop a universal application for downloading Chuck Norris jokes in [JSON](#) format.

Information

JSON (JavaScript Object Notation) is a text-based open standard designed for human-readable data interchange. It originates from the JavaScript scripting language to represent simple data structures and associative arrays called objects.

- [Source: Wikipedia](#)

Start

1. Launch the Visual Studio integrated development environment.
2. Create a new project: **Visual C# > Windows Universal > Blank App (Universal Windows)**.
3. Name the project: **JSON Reader 1.0**.

Add additional packages to the project by installing `Newtonsoft.Json` and `AngleSharp` from: **Tools > NuGet Package Manager > Package Manager Console**, running the following commands in the console:

```
PM> Install-Package Newtonsoft.Json -Version 13.0.1
PM> Install-Package AngleSharp -Version 0.16.1
```

Root.cs

Add a new class `Root.cs` that will be used to deserialize the data returned by the service.

```
public class Root
{
    public List<object> categories { get; set; }
    public string created_at { get; set; }
    public string icon_url { get; set; }
    public string id { get; set; }
    public string updated_at { get; set; }
    public string url { get; set; }
    public string value { get; set; }
}
```

Note

1. Load and copy an example JSON from:
<https://api.chucknorris.io/jokes/random>
2. Generate the C# class for the selected JSON from:
<http://json2csharp.com/>

MainPage.xaml

The **MainPage.xaml** file contains the source code for the application's user interface design and is written in XAML. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application.

```
<!-- User Interface (UI): JSON Reader 1.0 -->
<StackPanel Background="LightSalmon" Padding="30">

    <!-- Title -->
    <TextBlock Text="JSON Reader 1.0" FontSize="40" Margin="10"/>

    <!-- Joke -->
    <TextBox Name="Joke" FontSize="20" TextWrapping="Wrap" Height="300" Margin="10" />

    <!-- Buttons -->
    <StackPanel Orientation="Horizontal">
        <Button Content="New joke" Width="120" FontSize="23" Padding="10" Margin="10" />
        <Button Content="Tell joke" Width="120" FontSize="23" Padding="10" Margin="10" />
    </StackPanel>

</StackPanel>
```

View of the user interface design (XAML) in Visual Studio while developing the application:

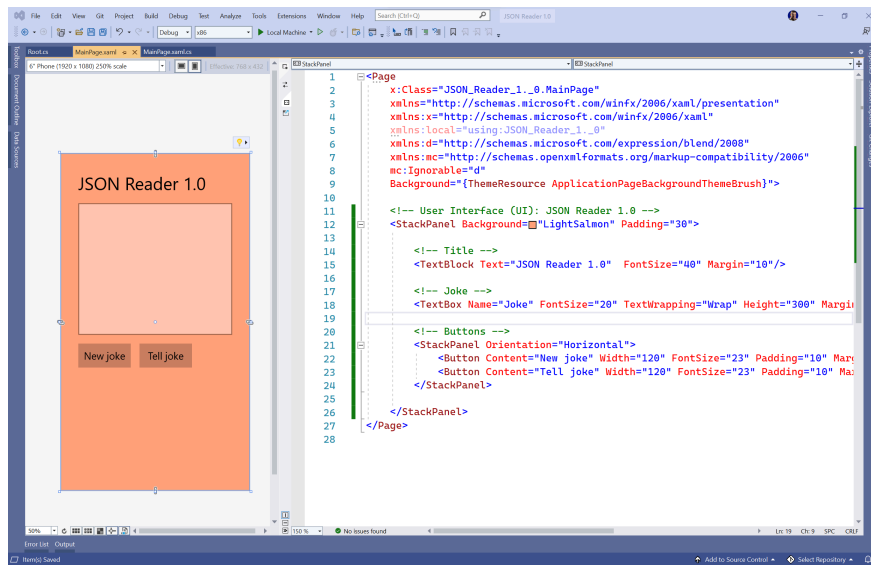


Fig. 1.47. View of the user interface design

MainPage.xaml.cs

The **MainPage.xaml.cs** file contains the source code for the application's business logic and is written in C#. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application.

```

// Business Logic (BL): JSON Reader 1.0
public sealed partial class MainPage : Page
{
    // Constructor
    public MainPage()
    {
        this.InitializeComponent();
    }

    // Button Get Joke Event Handler
    private async void Button_Get_Click(object sender, RoutedEventArgs e)
    {
        // Download JSON
        HttpClient client = new HttpClient();
        var json = await client.GetStringAsync(new Uri("https://api.chucknorris.io

        // Deserialize the JSON
        var joke = JsonConvert.DeserializeObject<Root>(json);

        // Parse the HTML
        var html = new HtmlParser().ParseDocument(joke.value);
        var text = html.Body.TextContent;

        // Tell the Joke
        Joke.Text = text;
    }

    // Button Tell Joke Event Handler
    private async void Button_Tell_Click(object sender, RoutedEventArgs e)
    {
        if (Joke.Text != "")
        {
            // The media object for controlling and playing audio.
            var mediaElement = new MediaElement();

            // The object for controlling the speech synthesis engine (voice).
            var synth = new SpeechSynthesizer();

            // Generate the audio stream from plain text.
            var stream = await synth.SynthesizeTextToStreamAsync(Joke.Text);

            // Send the stream to the media object.
            mediaElement.SetSource(stream, stream.ContentType);
            mediaElement.Play();
        }
    }
}

```

```

    }
}

```

View of the business logic (C#) in Visual Studio while developing the application:

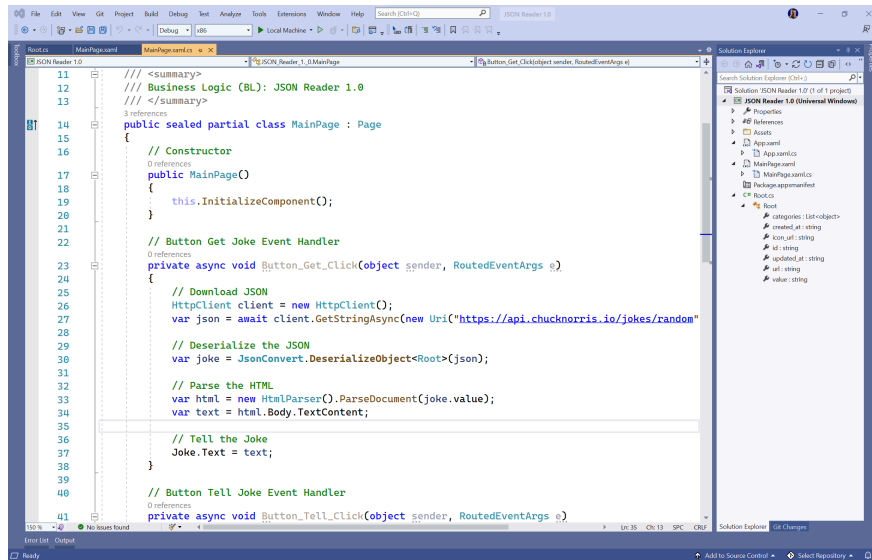


Fig. 1.48. View of the application's business logic

Demo

Run the application from the menu: **Debug > Start Debugging** or by pressing the **F5** key.

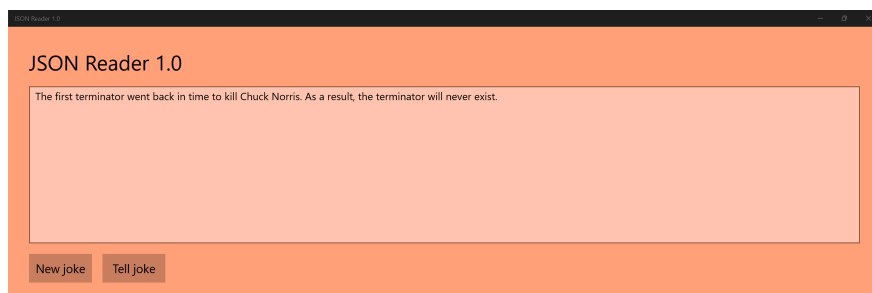


Fig. 1.49 Universal application for downloading Chuck Norris jokes

Chapter 2. Xamarin.Forms

Information

This chapter introduces the reader to the exciting world of Microsoft technologies for developing **Xamarin.Forms** applications.

Tip

Wishing you lots of fun, failures, and successes while using Microsoft technologies for designing and developing cross-platform applications.

Note

The examples in this section were developed using the following versions of the operating system and integrated development environment:

- Microsoft Windows 11 Version 10.0.22000.282
- Microsoft Visual Studio Community 2022 Version 17.0.0

History

Founded in May 2011, Xamarin provides developers with a unified platform for building cross-platform mobile applications. The engineering team—authors of Mono, Mono for Android, and MonoTouch—joined forces to create a unique product that, using Microsoft technologies C# and .NET, enables development for Android, iOS, and Windows applications.



Fig. 2.50. Cross-platform mobile applications

In February 2016 Microsoft announced the acquisition of Xamarin. The Redmond technology giant added the technology to the Visual Studio integrated development environment. As of April 2017, more than 1.4 million developers from 120 countries used Xamarin.

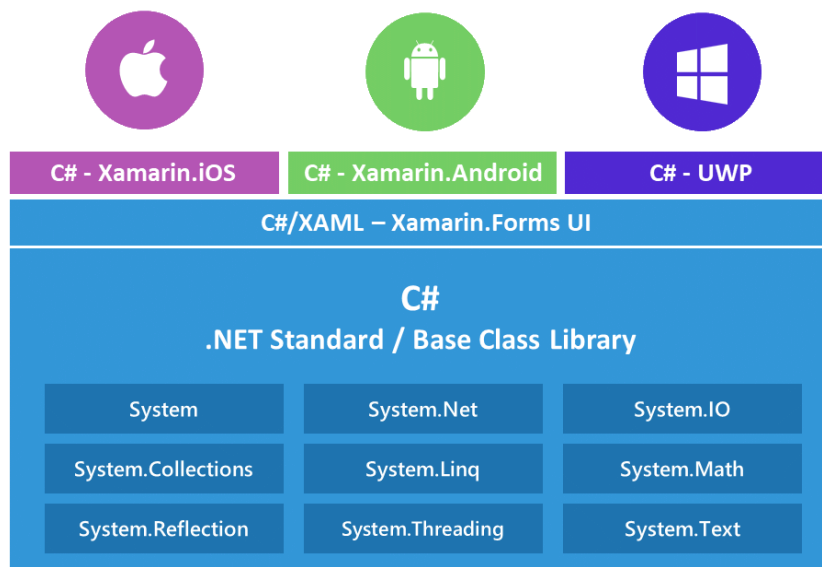


Fig. 2.51 Platform-dependent and shared source code

Applications

This section presents several applications demonstrating the development of **Xamarin.Forms** apps.

Creating each application in Visual Studio begins by creating a new project. Create a new project from the menu using: **File > New > Project**. You can also use the shortcut **Ctrl + Shift + N** to create a new project.

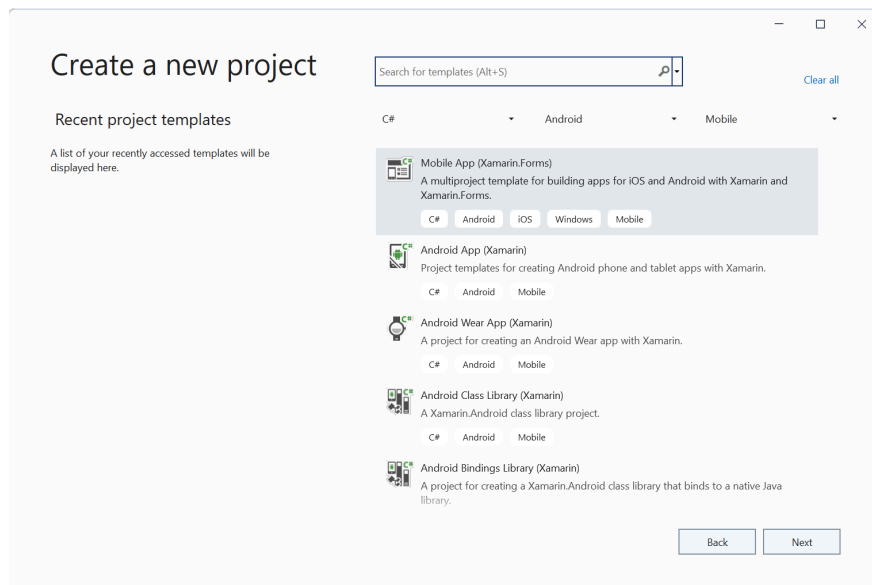


Fig. 2.52 Creating a new cross-platform project

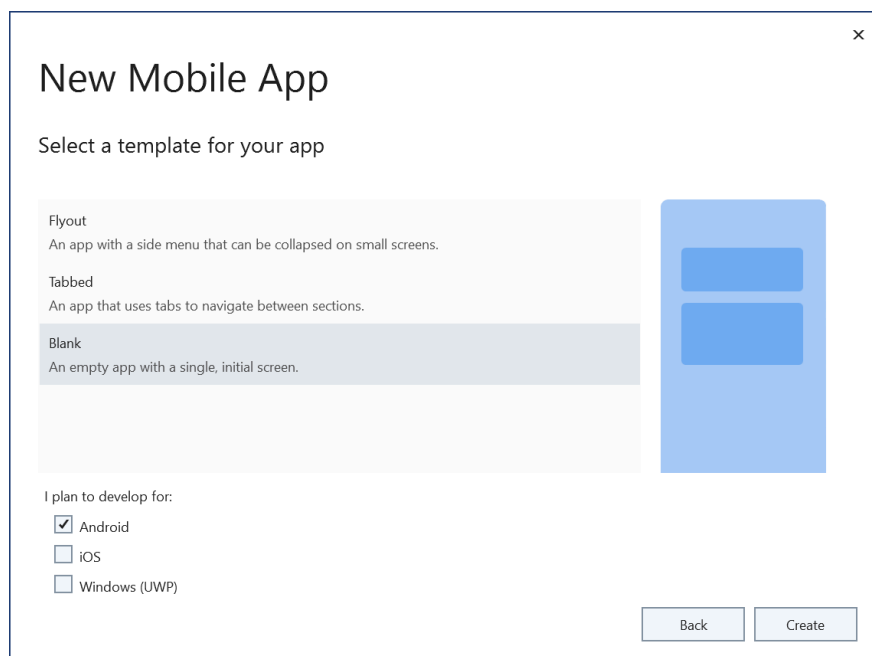


Fig. 2.53 Configuring the new cross-platform project

Settings

From **Tools > Android > Android Device Manager**, add a new emulator device for testing the applications under development:

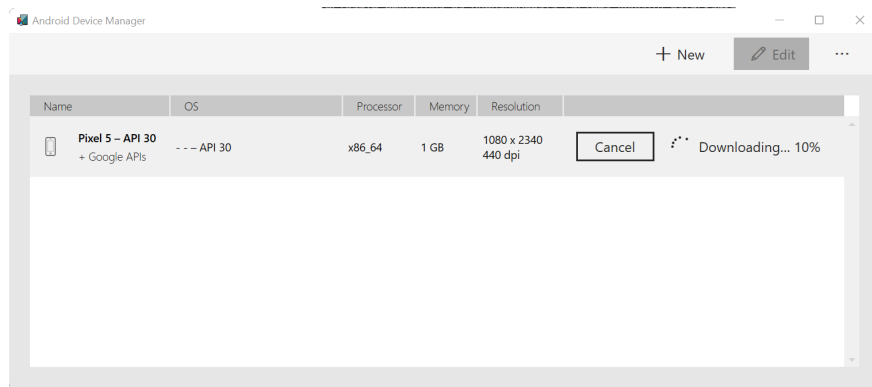


Fig. 2.54 Adding a new emulator device for testing applications.

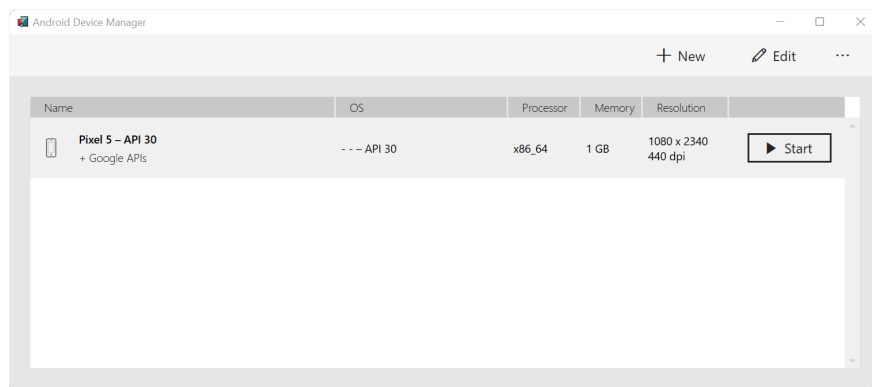


Fig. 2.55 A new emulator device has been successfully added for testing applications.

Test the Android emulator by pressing the **Start** button. If everything is correct you will see Android 11 (API 30), as shown in the figure below:



Fig. 2.56. Android 11 (API 30)

Tip

You can speed up the Android emulator on Microsoft Windows by using [HAXM](#) (Intel) or [WHPX](#) (AMD) depending on the CPU available in your computer.

Suma 2.0

Using the Visual Studio integrated development environment and the C# programming language, we will develop a cross-platform mobile application to find the sum of two numbers.

Start

1. Launch the Visual Studio integrated development environment.
2. Create a new project: **Visual C# > Cross-Platform > Mobile App (Xamarin.Forms)**.
3. Name the project: **Suma 2.0**.

MainPage.xaml

The **MainPage.xaml** file contains the source code for the application's user interface design and is written in XAML. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application.

```
<!-- User Interface (UI): Suma 2.0 -->
<StackLayout Padding="20">

    <!-- Title -->
    <Label Text="Suma 2.0" FontSize="Large" />

    <!-- A -->
    <Label Text="A=" />
    <Entry x:Name="boxA" Keyboard="Numeric" />

    <!-- B -->
    <Label Text="B=" />
    <Entry x:Name="boxB" Keyboard="Numeric" />

    <!-- A+B -->
    <Label Text="A+B=" />
    <Entry x:Name="boxAB" Keyboard="Numeric" />

    <!-- Button -->
    <Button Text="Sum" Clicked="OnButtonClicked" />

</StackLayout>
```

MainPage.xaml.cs

The **MainPage.xaml.cs** file contains the source code for the application's business logic and is written in C#. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application.

```
// Business Logic (BL): Suma 2.0

public partial class MainPage : ContentPage
{
    // Constructor
    public MainPage()
    {
        InitializeComponent();
    }

    // Button Click Event Handler
    void OnButtonClicked(object sender, EventArgs args)
    {
        var A = int.Parse(this.boxA.Text);
        var B = int.Parse(this.boxB.Text);
        var C = A + B;
        boxAB.Text = C.ToString();
    }
}
```

Demo

Run the application from the menu: **Debug > Start Debugging** or by pressing the **F5** key.

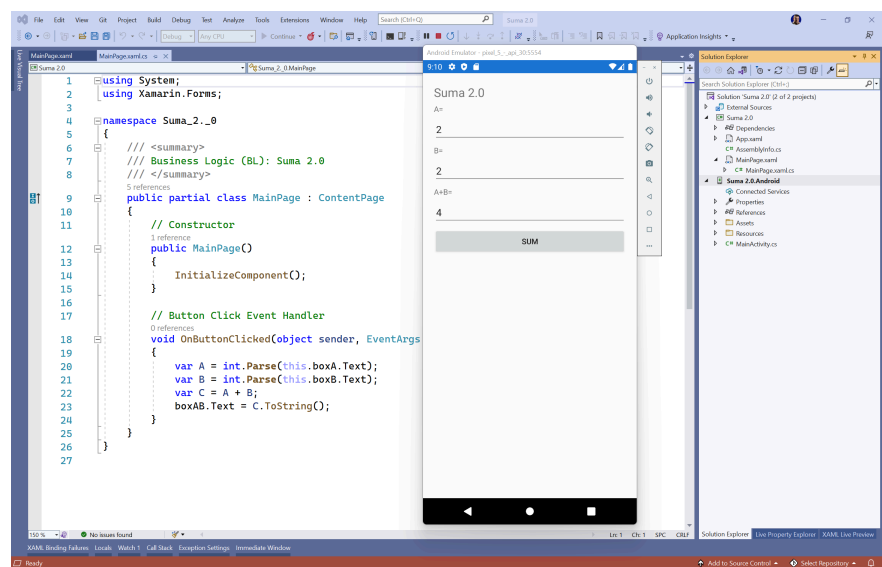


Fig. 2.57 Testing the cross-platform mobile application for finding the sum of two numbers - Android Emulator 11 (API 30)

Fibonacci 2.0

Using the Visual Studio integrated development environment and the C# programming language, we will develop a cross-platform mobile application to generate Fibonacci sequence numbers.

Information

In mathematics, the Fibonacci numbers form a sequence defined recursively as follows: it starts with 0 and 1, and each subsequent term is the sum of the previous two. The first Fibonacci numbers are: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, ...

- [Source: Wikipedia](#)

Start

1. Launch the Visual Studio integrated development environment.
2. Create a new project: **Visual C# > Cross-Platform > Mobile App (Xamarin.Forms)**.
3. Name the project: **Fibonacci 2.0**.

MainPage.xaml

The **MainPage.xaml** file contains the source code for the application's user interface design and is written in XAML. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application

```
<!-- User Interface (UI): Fibonacci 2.0 -->
<StackLayout Padding="20">

    <!-- Title -->
    <Label Text="Fibonacci 2.0" FontSize="Large" />

    <!-- Limit -->
    <Label Text="Limit" />
    <Entry x:Name="boxLimit" Keyboard="Numeric" Text="1000" />
    <Button Text="Generate" Clicked="OnButtonClicked" />

    <!-- Numbers -->
    <ListView x:Name="boxNumbers" />

</StackLayout>
```

MainPage.xaml.cs

The **MainPage.xaml.cs** file contains the source code for the application's business logic and is written in C#. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application.

```
// Business Logic (BL): Fibonacci 2.0
public partial class MainPage : ContentPage
{
    // Constructor
    public MainPage()
    {
        InitializeComponent();
    }

    // Button Click Event Handler
    void OnButtonClicked(object sender, EventArgs args)
    {
        List<int> numbers = new List<int>();
        numbers.Add(1);
        numbers.Add(1);
        int limit = int.Parse(this.boxLimit.Text);
        int a = 1, b = 1, c = a + b;
        while (c < limit)
        {
            numbers.Add(c);
            a = b;
            b = c;
            c = a + b;
        }
        this.boxNumbers.ItemsSource = numbers;
    }
}
```

Demo

Run the application from the menu: **Debug > Start Debugging** or by pressing the **F5** key.

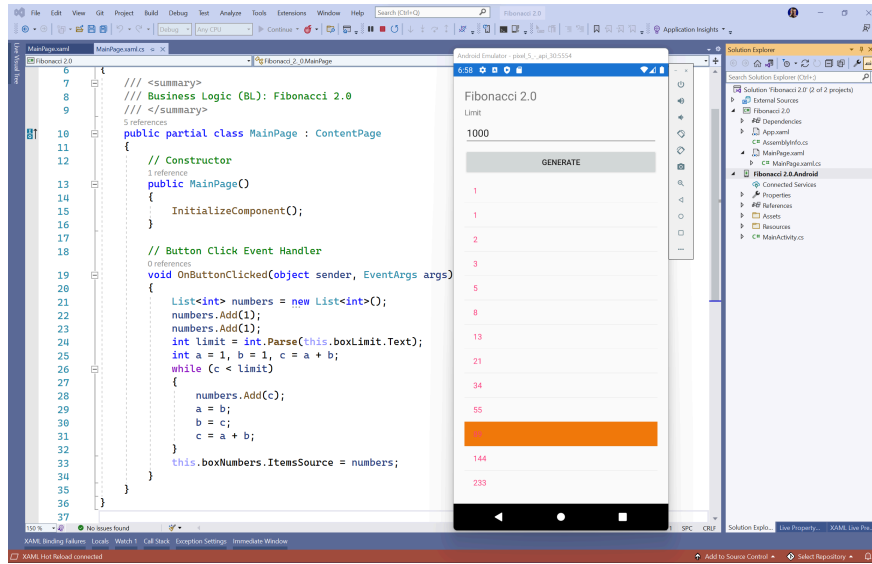


Fig. 2.58 Testing the cross-platform mobile application for generating the Fibonacci sequence - Android Emulator 11 (API 30)

Primes 2.0

Using the Visual Studio integrated development environment and the C# programming language, we will develop a cross-platform mobile application to generate a sequence of prime numbers.

Information

A prime number is a natural number greater than 1 that is not a product of two smaller natural numbers.

- Source: [Wikipedia](#)

Start

1. Launch the Visual Studio integrated development environment.
2. Create a new project: **Visual C# > Cross-Platform > Mobile App (Xamarin.Forms)**.
3. Name the project: **Primes 2.0**.

MainPage.xaml

The **MainPage.xaml** file contains the source code for the application's user interface design and is written in XAML. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application

```
<!-- User Interface (UI): Primes 2.0 -->
<StackLayout Padding="20" BackgroundColor="Yellow">

    <!-- Title -->
    <Label Text="Primes 2.0" FontSize="Large" />

    <!-- Limit -->
    <Label Text="Limit" />
    <Entry x:Name="boxLimit" Keyboard="Numeric" Text="1000" />
    <Button Text="Generate" Clicked="OnButtonClicked" />

    <!-- Numbers -->
    <ListView x:Name="boxNumbers" />

</StackLayout>
```

MainPage.xaml.cs

The **MainPage.xaml.cs** file contains the source code for the application's business logic and is written in C#. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application.

```
// Business Logic (BL): Primes 2.0
public partial class MainPage : ContentPage
{
    // Constructor
    public MainPage()
    {
        InitializeComponent();
    }

    // Button Click Event Handler
    void OnButtonClicked(object sender, EventArgs args)
    {
        List<int> primes = new List<int>();
        int limit = int.Parse(this.boxLimit.Text);
        for (int k = 2; k < limit; k++)
        {
            bool prime = true;
            for (int j = 2; j < k; j++) if (k % j == 0) prime = false;
            if (prime) primes.Add(k);
        }
        this.boxNumbers.ItemsSource = primes;
    }
}
```

Demo

Run the application from the menu: **Debug > Start Debugging** or by pressing the **F5** key.

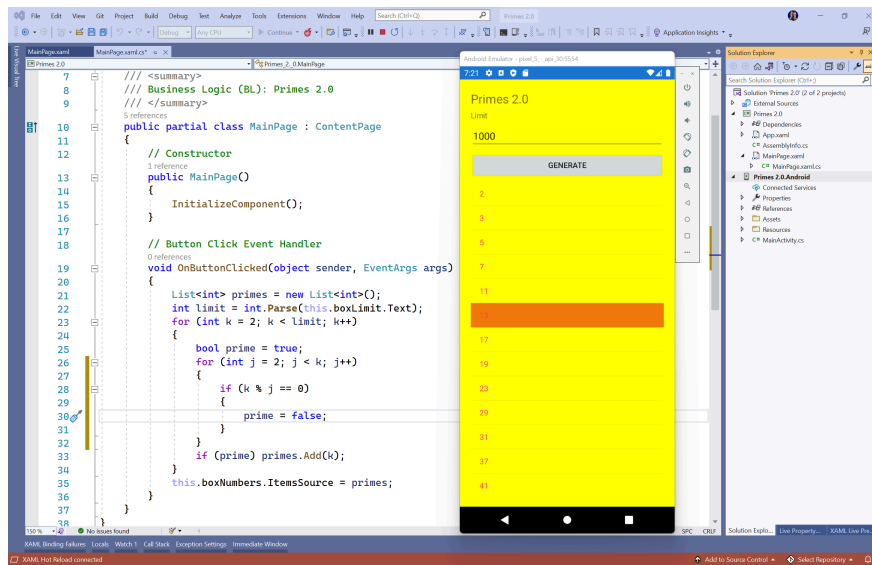


Fig. 2.59 Testing the cross-platform mobile application for generating a sequence of prime numbers - Android Emulator 11 (API 30)

Phone Book 2.0

Using the Visual Studio integrated development environment and the C# programming language, we will develop a cross-platform mobile application for a phone book containing a list of contacts.

Start

1. Launch the Visual Studio integrated development environment.
2. Create a new project: **Visual C# > Cross-Platform > Mobile App (Xamarin.Forms)**.
3. Name the project: **Phone Book 2.0**.

Add additional packages to the project by installing `Newtonsoft.Json` from: **Tools > NuGet Package Manager > Package Manager Console**, by running the following command in the console:

```
PM> Install-Package Newtonsoft.Json -Version 13.0.1
```

Contact.cs

Add a new class `Contact.cs` containing the following code fragment:

```

public class Contact
{
    private Uri picture;

    public Uri Picture
    {
        get { return picture; }
        set { picture = value; }
    }

    private string name;

    public string Name
    {
        get { return name; }
        set { name = value; }
    }

    private string phone;

    public string Phone
    {
        get { return phone; }
        set { phone = value; }
    }

    public Contact(Uri picture, string name, string phone)
    {
        this.picture = picture;
        this.name = name;
        this.phone = phone;
    }
}

```

ViewModel.cs

Add a new class `ViewModel.cs` containing the following code fragment:


```

public static class ViewModel
{
    public static ObservableCollection<Contact> Contacts;

    static ViewModel()
    {
        Contacts = new ObservableCollection<Contact>();

        BindingBase.EnableCollectionSynchronization(Contacts, null, Callback);
    }

    private static void Callback(IEnumerable collection, object context, Action action)
    {
        lock (collection)
        {
            accessMethod?.Invoke();
        }
    }
}

```

MainPage.xaml

The **MainPage.xaml** file contains the source code for the application's user interface design and is written in XAML. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application.

```

<!-- User Interface (UI): Phone Book 2.0 -->
<StackLayout Padding="50">

    <Label Text="Phone Book 2.0" FontSize="Large" FontAttributes="Bold" />

    <Button Text="Add" Clicked="Button_Clicked" />

    <ListView x:Name="ListView">
        <ListView.ItemTemplate>
            <DataTemplate>
                <ImageCell Text="{Binding Name}"
                    Detail="{Binding Phone}"
                    ImageSource="{Binding Picture}" />
            </DataTemplate>
        </ListView.ItemTemplate>
    </ListView>

</StackLayout>

```

MainPage.xaml.cs

The **MainPage.xaml.cs** file contains the source code for the application's business logic and is written in C#. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application.

```
// Business Logic (BL): Phone Book 2.0
public partial class MainPage : ContentPage
{
    public MainPage(Contact contact = null)
    {
        InitializeComponent();

        this.ListView.ItemsSource = ViewModel.Contacts;

        if (contact is Contact)
        {
            ViewModel.Contacts.Add(contact);
        }
    }

    private void Button_Clicked(object sender, System.EventArgs e)
    {
        Navigation.PushModalAsync(new AddPage());
    }
}
```

AddPage.xaml

Add a new page `AddPage.xaml` . Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application.

```

<!-- User Interface (UI): Phone Book 2.0 -->
<StackLayout Padding="50">

    <Label Text="Add" FontSize="Large" FontAttributes="Bold" />

    <Label Text="Picture" FontSize="Large" />
    <Entry x:Name="Picture" FontSize="Large" />

    <Label Text="Name" FontSize="Large" />
    <Entry x:Name="Name" FontSize="Large" />

    <Label Text="Phone" FontSize="Large" />
    <Entry x:Name="Phone" FontSize="Large" />

    <Button Text="Save" Clicked="Button_Clicked" />

</StackLayout>

```

AddPage.xaml.cs

Open the file **AddPage.xaml.cs**. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application.

```

// Business Logic (BL): Phone Book 2.0
public partial class AddPage : ContentPage
{
    public AddPage()
    {
        InitializeComponent();
    }

    private void Button_Clicked(object sender, System.EventArgs e)
    {
        var contact = new Contact
        (
            picture: new Uri(this.Picture.Text),
            name: this.Name.Text,
            phone: this.Phone.Text
        );

        Navigation.PushModalAsync(new MainPage(contact));
    }
}

```

Demo

Run the application from the menu: **Debug > Start Debugging** or by pressing the **F5** key.

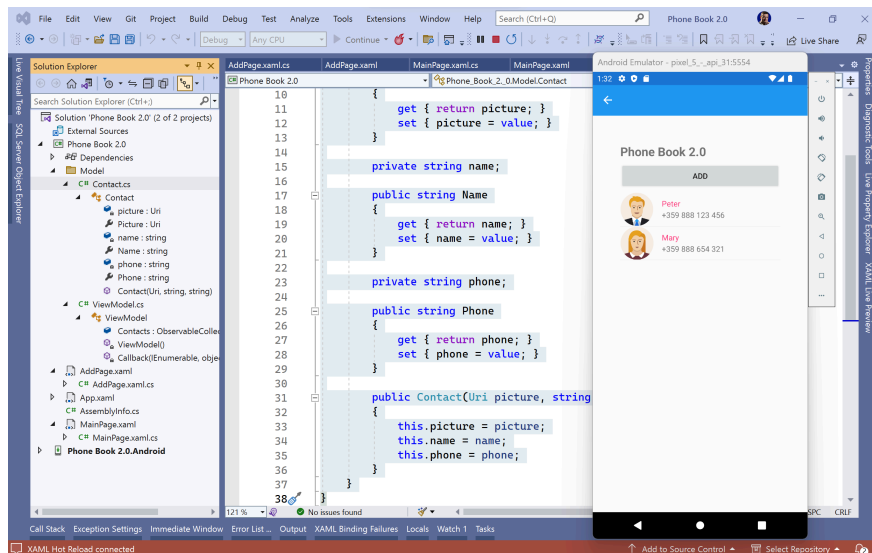


Fig. 2.64. Demonstration of the cross-platform mobile phone book application

Used images for user profiles

1. Male: <https://icons-for-free.com/business+costume+male+man+office+user+icon-1320196264882354682.png>
2. Female: <https://icons-for-free.com/female+person+user+woman+young+icon-1320196266256009072.png>

Clicker Mania 3.0

Using the Visual Studio integrated development environment and the C# programming language, we will develop a cross-platform mobile application that counts the user's clicks during a given time.

Start

1. Launch the Visual Studio integrated development environment.
2. Create a new project: **Visual C# > Cross-Platform > Mobile App (Xamarin.Forms)**.
3. Name the project: **Clicker Mania 3.0**.

MainPage.xaml

The **MainPage.xaml** file contains the source code for the application's user interface design and is written in XAML. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application

```
<!-- User Interface (UI): Clicker Mania 3.0 -->
<StackLayout Padding="20" BackgroundColor="Lime">

    <!-- Title -->
    <Label Text="Clicker Mania 3.0" FontSize="Large" />

    <!-- Timer -->
    <Label Text="Timer" />
    <Entry x:Name="Timer" Text="0" />

    <!-- Clicks -->
    <Label Text="Clicks" />
    <Entry x:Name="Clicks" Text="0" />

    <!-- Clicks Per Minute -->
    <Label Text="Clicks Per Minute" />
    <Entry x:Name="CPM" Text="0" />

    <!-- Button -->
    <Button Text="Click" Clicked="OnButtonClicked" />

</StackLayout>
```

MainPage.xaml.cs

The **MainPage.xaml.cs** file contains the source code for the application's business logic and is written in C#. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application.

```
// Business Logic (BL): Clicker Mania 3.0
public partial class MainPage : ContentPage
{
    // Constructor
    public MainPage()
    {
        this.InitializeComponent();

        // Timer
        Device.StartTimer(TimeSpan.FromSeconds(1), TimerTick);
    }

    // Timer Tick Event Handler
    private bool TimerTick()
    {
        int T = int.Parse(this.Timer.Text);
        this.Timer.Text = (++T).ToString();

        // Clicks Per Minute
        this.CPM.Text = (float.Parse(this.Clicks.Text) /
            float.Parse(this.Timer.Text) * 60).ToString("N2");
        return true;
    }

    // Button Click Event Handler
    void OnButtonClicked(object sender, EventArgs args)
    {
        int N = int.Parse(this.Clicks.Text);
        this.Clicks.Text = (++N).ToString();
    }
}
```

Demo

Run the application from the menu: **Debug > Start Debugging** or by pressing the **F5** key.

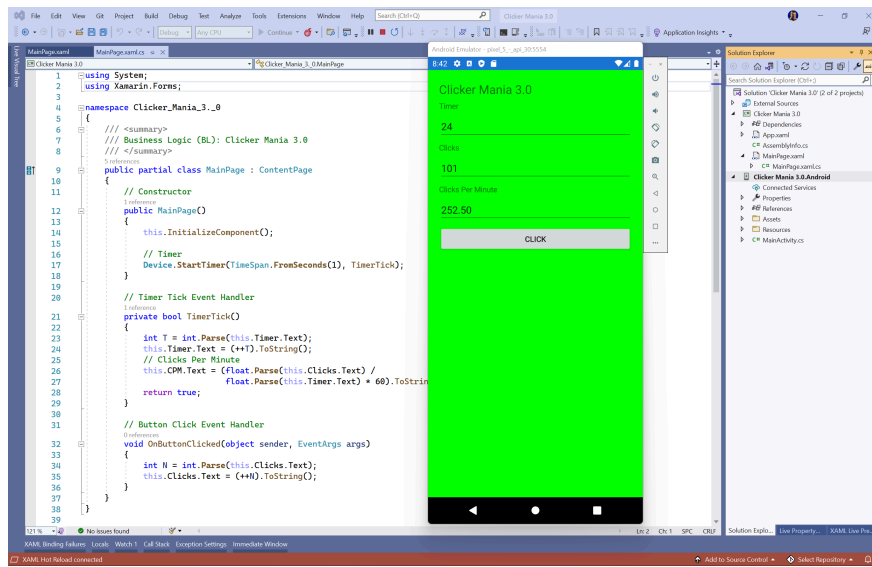


Fig. 2.60 Testing the cross-platform mobile application that counts user clicks during a given time - Android Emulator 11 (API 30)

HTML Downloader 2.0

Using the Visual Studio integrated development environment and the C# programming language, we will develop a cross-platform mobile application to download the HTML content of a web page.

Information

HTML is the standard markup language for describing and designing web pages. HTML is a standard on the Internet and its rules are defined by the World Wide Web Consortium (W3C).

- Source: [Wikipedia](#)

Start

1. Launch the Visual Studio integrated development environment.
2. Create a new project: **Visual C# > Cross-Platform > Mobile App (Xamarin.Forms)**.
3. Name the project: **HTML Downloader 2.0**.

Install the additional package for the app from: **Tools > NuGet Package Manager > Package Manager Console**, by running the following command in the console:

```
PM> Install-Package AngleSharp -Version 0.16.1
```

MainPage.xaml

The **MainPage.xaml** file contains the source code for the application's user interface design and is written in XAML. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application.


```

<!-- User Interface (UI): HTML Downloader 2.0 -->
<StackLayout Padding="20" BackgroundColor="LightCoral">

    <!-- Title -->
    <Label Text="HTML Downloader 2.0" FontSize="Large" />

    <!-- URL -->
    <Label Text="URL" FontSize="Large" />
    <Entry x:Name="URL" Text="https://www.minchev.eu" />
    <Button Text="Download" Clicked="OnButtonClicked" />

    <!-- HTML -->
    <ScrollView>
        <Label x:Name="HTML" />
    </ScrollView>

</StackLayout>

```

MainPage.xaml.cs

The **MainPage.xaml.cs** file contains the source code for the application's business logic and is written in C#. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application.

```
// Business Logic (BL): HTML Downloader 2.0
public partial class MainPage : ContentPage
{
    // Constructor
    public MainPage()
    {
        InitializeComponent();
    }

    // Button Click Event Handler
    private async void OnButtonClicked(object sender, EventArgs args)
    {
        // Get HTML
        string html = await Download(new Uri(this.URL.Text));

        // AngleSharp HTML to Text Parser
        var temp = new HtmlParser().ParseDocument(html);
        string text = temp.Body.TextContent;

        // Plain Text
        this.HTML.Text = text;
    }

    // Download Handler
    private async Task<string> Download(Uri link)
    {
        HttpClient client = new HttpClient();
        return await client.GetStringAsync(link);
    }
}
```

Demo

Run the application from the menu: **Debug > Start Debugging** or by pressing the **F5** key.

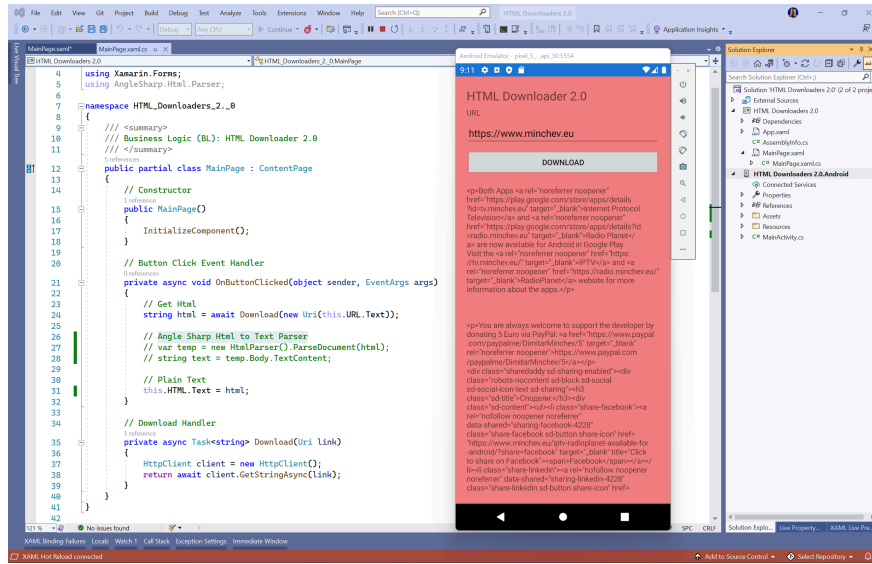


Fig. 2.61 Testing the cross-platform mobile application for downloading the HTML content of a web page - Android Emulator 11 (API 30).

RSS Reader 2.0

Using the Visual Studio integrated development environment and the C# programming language, we will develop a cross-platform mobile application for reading news feeds (RSS) from internet sources.

Information

RSS is a software mechanism for exchanging news between websites or between a site and a user. It represents a set of formats for delivering information from the World Wide Web.

- Source: [Wikipedia](#)

Start

1. Launch the Visual Studio integrated development environment.
2. Create a new project: **Visual C# > Cross-Platform > Mobile App (Xamarin.Forms)**.
3. Name the project: **RSS Reader 2.0**.

FeedReader.cs

Add a new class named `FeedReader.cs`, which will be used to store data from the news feed.

```

/// <summary>
/// RSS Feed Item
/// </summary>
public class FeedItem
{
    public string Title { get; set; }
    public string Description { get; set; }
    public string Link { get; set; }
    public string PubDate { get; set; }
}

/// <summary>
/// RSS Feed Reader
/// </summary>
public class FeedReader
{
    public IEnumerable<FeedItem> ReadFeed(string url)
    {
        var feed = XDocument.Load(url);
        var posts = from item in feed.Descendants("item")
                    select new FeedItem
                    {
                        Title = item.Element("title").Value,
                        Description = item.Element("description").Value,
                        Link = item.Element("link").Value,
                        PubDate = item.Element("pubDate").Value
                    };
        return posts;
    }
}

```

HyperlinkButton.cs

Add a `HyperlinkButton.cs` class that will be used to add hyperlinks to the application's user interface, following Microsoft documentation: [Xamarin.Forms Hyperlinks Microsoft Docs](#)

```

/// <summary>
/// Hyperlink Button Control
/// </summary>
public class HyperlinkButton : TextCell
{
    // Url Property
    public static readonly BindableProperty UrlProperty = BindableProperty.Create(
        nameof(Url), typeof(string), typeof(HyperlinkButton), null);

    public string Url
    {
        get { return (string)GetValue(UrlProperty); }
        set { SetValue(UrlProperty, value); }
    }

    // Constructor
    public HyperlinkButton()
    {
        Tapped += HyperlinkButton_Tapped;
    }

    // Tapped Event Handler
    private void HyperlinkButton_Tapped(object sender, System.EventArgs e)
    {
        // Launcher.OpenAsync is provided by Xamarin.Essentials.
        Command = new Command(async () => await Launcher.OpenAsync(Url));
    }
}

```

MainPage.xaml

The **MainPage.xaml** file contains the source code for the application's user interface design and is written in XAML. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application.

```

<!-- User Interface (UI): RSS Reader 2.0 -->
<StackLayout Padding="20" BackgroundColor="LightBlue">

    <!-- Title -->
    <Label Text="RSS Reader 2.0" FontSize="Large" />

    <!-- URI -->
    <Entry x:Name="URI" Text="https://www.minchev.eu/feed/" />
    <Button Text="Download" Clicked="Button_Clicked" />

    <!-- RSS -->
    <ListView x:Name="RSS">
        <ListView.ItemTemplate>
            <DataTemplate>
                <local:HyperlinkButton Text="{Binding Title}" Detail="{Binding Pub
            </DataTemplate>
        </ListView.ItemTemplate>
    </ListView>
</StackLayout>

```

MainPage.xaml.cs

The **MainPage.xaml.cs** file contains the source code for the application's business logic and is written in C#. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application.

```

// Business Logic (BL): RSS Reader 2.0
public partial class MainPage : ContentPage
{
    // Constructor
    public MainPage()
    {
        InitializeComponent();
    }

    // Button Click Event Handler
    private void Button_Clicked(object sender, EventArgs e)
    {
        RSS.ItemsSource = new ObservableCollection<FeedItem>
        (
            new FeedReader().ReadFeed(URI.Text)
        );
    }
}

```

Demo

Run the application from the menu: **Debug > Start Debugging** or by pressing the **F5** key.

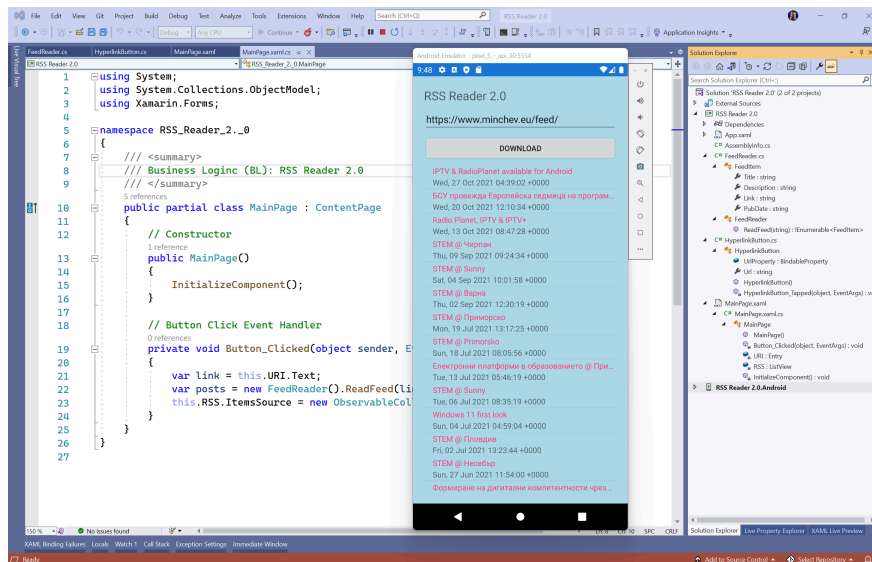


Fig. 2.62 Testing the cross-platform mobile application for reading internet news feeds (RSS sources) - Android Emulator 11 (API 30).

JSON Reader 2.0

Using the Visual Studio integrated development environment and the C# programming language, we will develop a cross-platform mobile application for downloading Chuck Norris jokes in [JSON](#) format.

Information

JSON (JavaScript Object Notation) is a text-based open standard designed for human-readable data interchange. It originates from the JavaScript scripting language to represent simple data structures and associative arrays called objects.

- [Source: Wikipedia](#)

Start

1. Launch the Visual Studio integrated development environment.
2. Create a new project: **Visual C# > Cross-Platform > Mobile App (Xamarin.Forms)**.
3. Name the project: **JSON Reader 2.0**.

Add additional packages to the project by installing `Newtonsoft.Json` from: **Tools > NuGet Package Manager > Package Manager Console**, by running the following command in the console:

```
PM> Install-Package Newtonsoft.Json -Version 13.0.1
```

Root.cs

Add a new class `Root.cs` that will be used to deserialize the data returned by the service.

```
public class Root
{
    public List<object> categories { get; set; }

    public string created_at { get; set; }

    public string icon_url { get; set; }

    public string id { get; set; }

    public string updated_at { get; set; }

    public string url { get; set; }

    public string value { get; set; }
}
```

Note

1. Load and copy an example JSON from:
<https://api.chucknorris.io/jokes/random>
2. Generate the C# class for the selected JSON from:
<http://json2csharp.com/>

MainPage.xaml

The **MainPage.xaml** file contains the source code for the application's user interface design and is written in XAML. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application.

```

<!-- User Interface (UI): JSON Reader 2.0 -->
<StackLayout Padding="50">

    <!-- Title -->
    <Label Text="JSON Reader 2.0" FontSize="Large" FontAttributes="Bold" />

    <!-- Buttons -->
    <StackLayout Orientation="Horizontal">
        <Button Text="New Joke" Clicked="NewJokeButtonClickedEventHandler" />
        <Button Text="Tell Joke" Clicked="TellJokeButtonClickedEventHandler" />
    </StackLayout>

    <!-- Joke -->
    <Label x:Name="JOKE" FontSize="Large" />

</StackLayout>

```

MainPage.xaml.cs

The **MainPage.xaml.cs** file contains the source code for the application's business logic and is written in C#. Copy (Ctrl+C) and paste (Ctrl+V) the fragment below into your application.

```

// Business Logic (BL): JSON Reader 2.0
public partial class MainPage : ContentPage
{
    /// <summary>
    /// Constructor
    /// </summary>
    public MainPage()
    {
        InitializeComponent();

        /// <summary>
        /// New Joke Button Clicked Event Handler
        /// </summary>
        private async void NewJokeButtonClickedEventHandler(object sender, EventArgs e)
        {
            // 1. Http Client
            HttpClient client = new HttpClient();

            // 2. Http Request to Receive JSON Response
            string json = await client.GetStringAsync(new Uri("https://api.chucknorris"));

            // 3. Deserialize JSON to Object
            Joke joke = JsonConvert.DeserializeObject<Joke>(json);

            // 4. Show the Joke in the UI
            JOKE.Text = joke.value;
        }

        /// <summary>
        /// Tell Joke Button Clicked Event Handler
        /// </summary>
        private async void TellJokeButtonClickedEventHandler(object sender, EventArgs e)
        {
            var joke = JOKE.Text;
            if (joke != "")
            {
                await TextToSpeech.SpeakAsync(joke);
            }
        }
    }
}

```

Demo

Run the application from the menu: **Debug > Start Debugging** or by pressing the **F5** key.

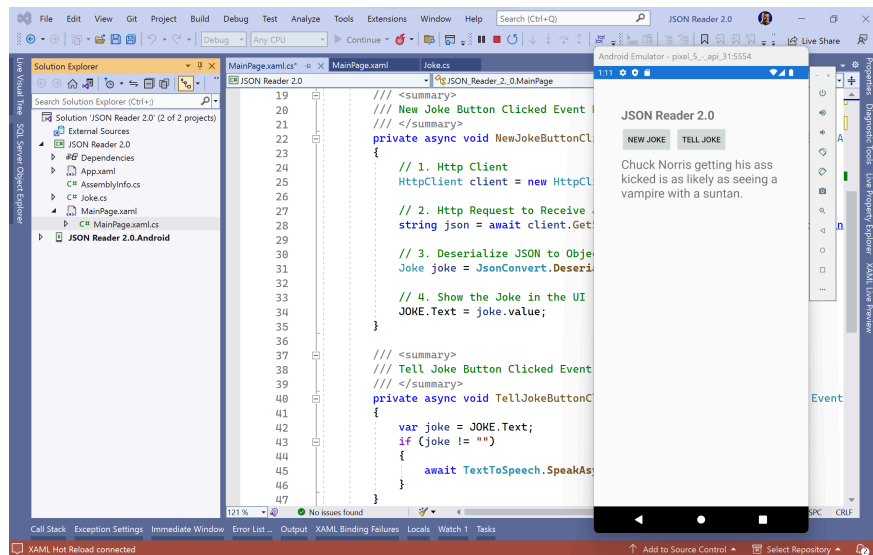


Fig. 2.63. Demonstration of the cross-platform mobile application for downloading Chuck Norris jokes

Chapter 3. WinUI

Information

This chapter introduces the reader to the exciting world of Microsoft technologies for developing **Windows UI Library** (WinUI) applications.

Tip

Wishing you lots of fun, failures, and successes while using Microsoft technologies for designing and developing cross-platform applications.

Note

The examples in this section were developed using the following versions of the operating system and integrated development environment:

- Microsoft Windows 11 Version 25H2 Build 26200
- Microsoft Visual Studio Community 2026 Version 18

History

WinUI (codename: Jupiter) was introduced as part of the Windows Runtime (WinRT) programming model, which was designed to support applications for the Universal Windows Platform (UWP) and aimed to provide a modern UI framework for Windows 8 and later.

With the release of Windows 10, **WinUI** became an integral part of UWP, allowing developers to build apps that run across a variety of Windows devices. This included enhancements to XAML that enabled more dynamic and responsive user interfaces.

WinUI 2

This version served as an extension library for UWP XAML, providing additional controls and styling aligned with the Windows 11 design language. It was distributed via NuGet and closely tied to the UWP framework.

WinUI 3

Released as part of the Windows App SDK (codename: Project Reunion), WinUI 3 separated the UI framework from the operating system, enabling faster updates and the introduction of new features without being tied to Windows updates. This version supports both desktop and UWP apps, allowing developers to gradually migrate existing applications to the new framework.

Fluent Design

WinUI embraces the Fluent Design System, providing a consistent and modern user experience across apps. It supports multiple programming languages including C++, C#, and JavaScript, making it versatile for developers.

Community and Support

WinUI libraries are actively maintained, with a dedicated GitHub repository for feature requests and bug reports. This community-driven approach helps ensure WinUI evolves in response to developers' needs.

Conclusion

WinUI represents a significant evolution in Windows app development, moving from a tightly coupled framework to a more flexible and modern UI library. Its history reflects Microsoft's ongoing efforts to streamline and improve the development experience for Windows applications, adapting to the changing software landscape.

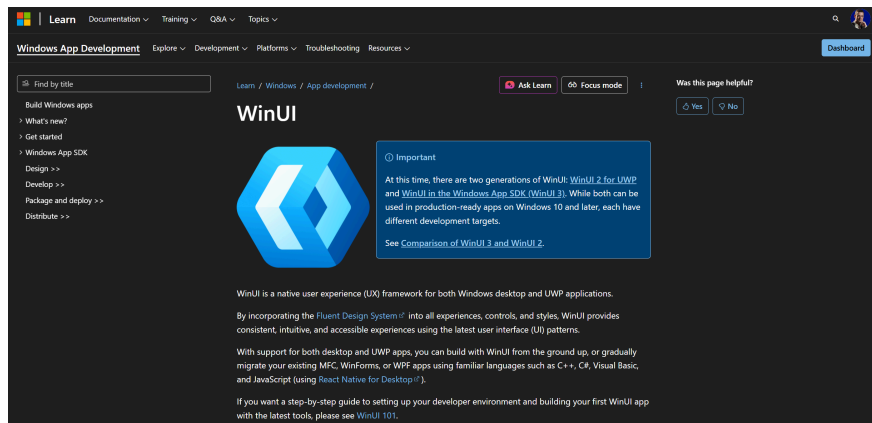


Fig. 3.1. WinUI App Development Documentation

For more information: [WinUI App Development Documentation](#).

Installation

You can download the free Community edition of the integrated development environment, **Microsoft Visual Studio Community Edition**, from the official website: <https://visualstudio.microsoft.com/>.

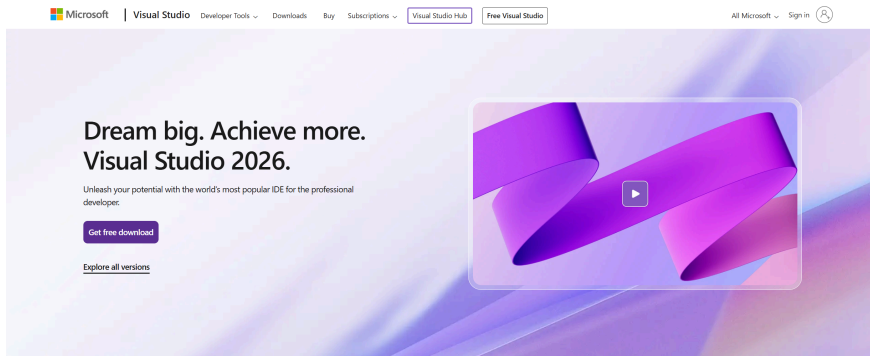


Fig. 3.2. Microsoft Visual Studio 2026 Website

For the development in this chapter we will use **Microsoft Visual Studio Community Edition 2026**; during installation be sure to select the **WinUI** application development workload/option.

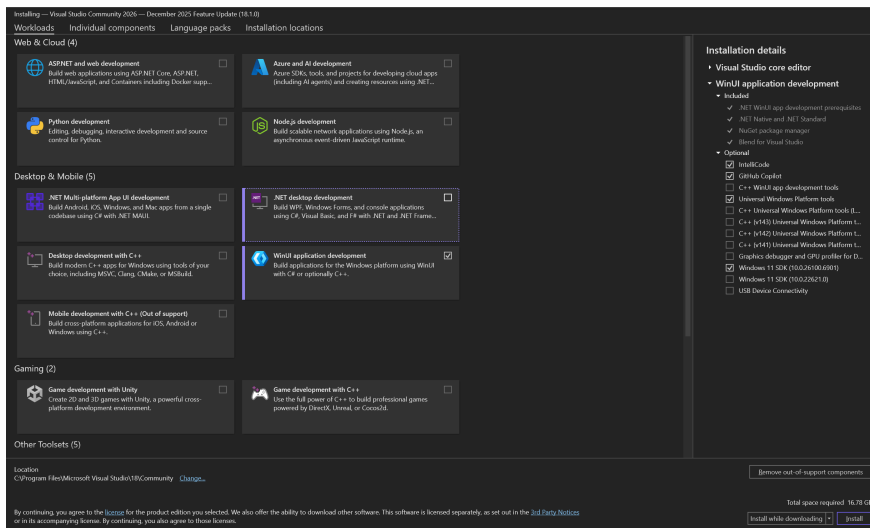


Fig. 3.3. WinUI workload selection during installation

Selecting additional developer tools increases the disk space required to install the IDE.

Proceed with the installation and wait for it to complete; the process may take a considerable amount of time.

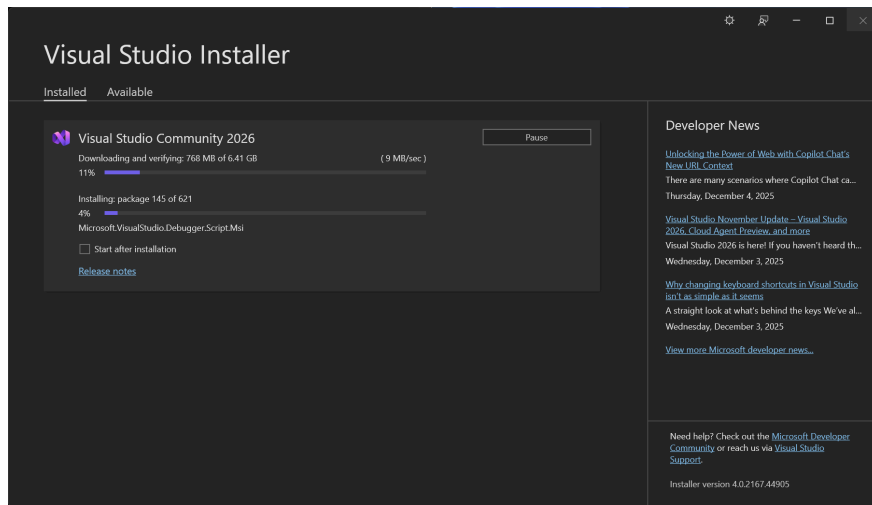


Fig. 3.4. Visual Studio Installation

Applications

Creating any application in the integrated development environment **Microsoft Visual Studio 2026** starts by creating a new project (Fig. 3.5).

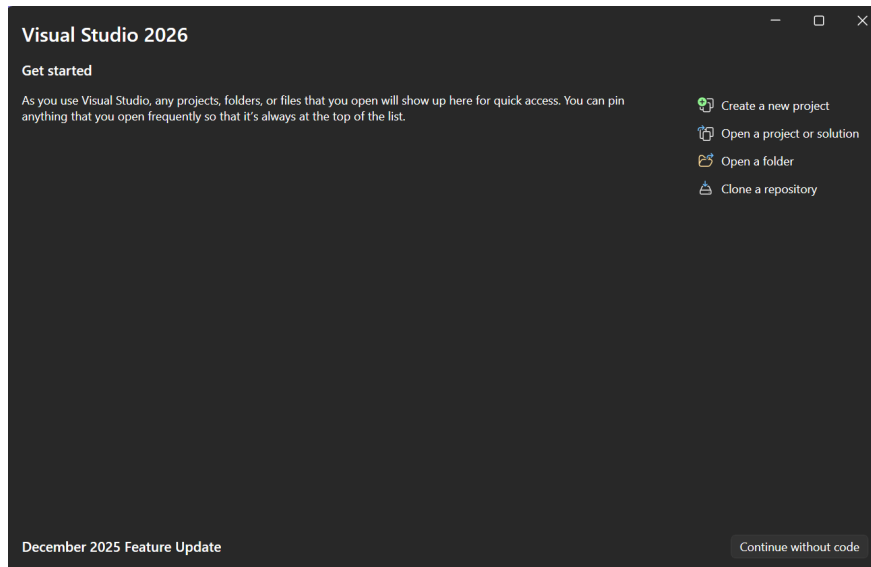


Fig. 3.5. Creating a new project in Visual Studio

On the next screen you choose the project type in **Microsoft Visual Studio 2026** (Fig. 3.6).

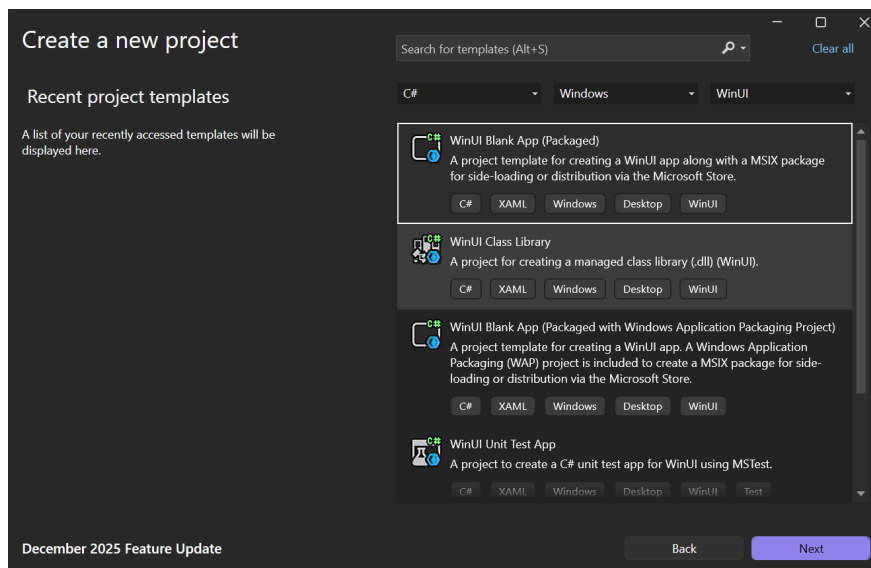


Fig. 3.6. Selecting the project type in Visual Studio

On the final screen you choose the project name and the location where it will be stored in **Microsoft Visual Studio 2026** (Fig. 3.7).

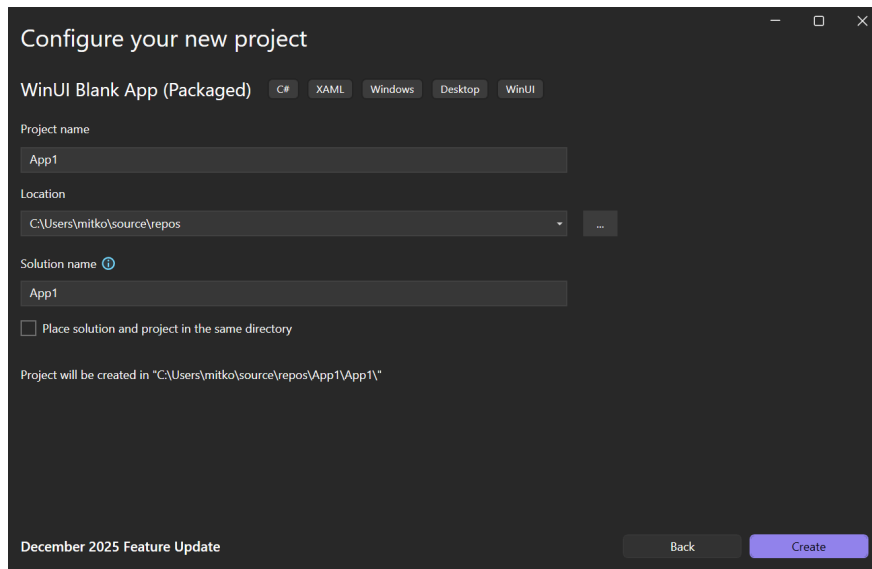


Fig. 3.7. Choosing the project name and storage location in Visual Studio

Suma 3.0

Start the integrated development environment **Microsoft Visual Studio 2026**, create a new project and name it **Suma 3.0**.

MainPage.xaml

This file contains the source code for the application's user interface design.

```
<!-- User Interface (UI): Suma 3.0 -->
<StackPanel Padding="50" Background="Orange">

    <!-- Title -->
    <TextBlock Text="Suma 3.0" FontSize="42" />

    <!-- A -->
    <TextBlock Text="A=" FontSize="24" />
    <TextBox x:Name="boxA" FontSize="24" />

    <!-- B -->
    <TextBlock Text="B=" FontSize="24" />
    <TextBox x:Name="boxB" FontSize="24" />

    <!-- A+B -->
    <TextBlock Text="A+B=" FontSize="24" />
    <TextBox x:Name="boxAB" FontSize="24" />

    <!-- Button -->
    <Button Content="Sum" FontSize="24"
        Margin="0 20 0 0" Padding="20 10 20 10"
        Click="OnButtonClicked" />

</StackPanel>
```

MainPage.xaml.cs

This file contains the source code for the application's business logic.

```
// Business Logic (BL): Suma 3.0
public sealed partial class MainWindow : Window
{
    // Constructor
    public MainWindow()
    {
        InitializeComponent();
    }

    // Button Click Event Handler
    private void OnButtonClicked(object sender, RoutedEventArgs e)
    {
        var A = int.Parse(this.boxA.Text);
        var B = int.Parse(this.boxB.Text);
        var C = A + B;
        this.boxAB.Text = C.ToString();
    }
}
```

Running the app

Run the application and test its functionality as shown in the figure below.

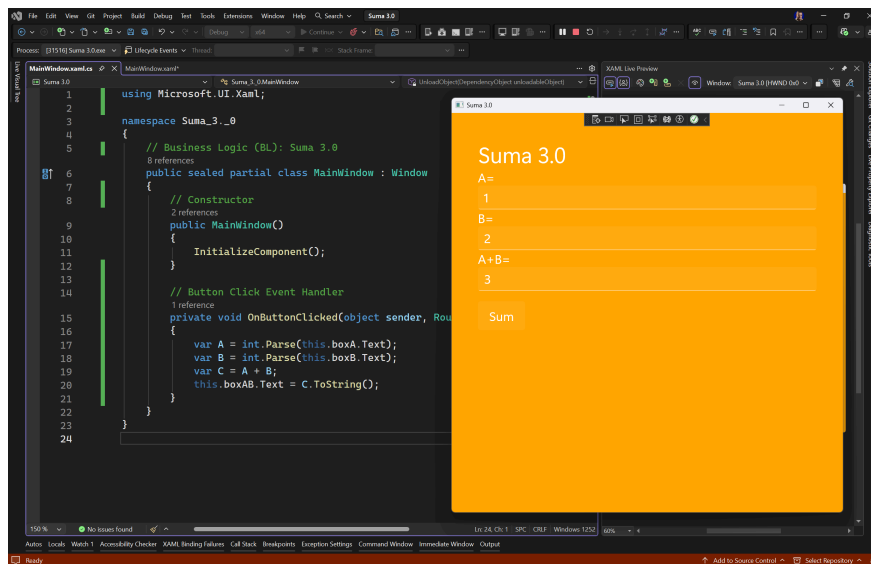


Fig. 3.8. Suma 3.0

Note

Do you notice similarities with the examples from Chapter 1? Try to implement all examples from Chapter 1 yourself, using the **Windows UI Library** (WinUI) as the project type.

Chapter 4. MAUI

Info

This chapter introduces the reader to the exciting world of Microsoft technologies for building Multi-Platform App UI (MAUI) applications.

Tip

Wishing you much fun, failures, and successes while using Microsoft technologies for designing and building cross-platform applications.

Note

The examples in this section were developed using the following OS and IDE versions:

- Microsoft Windows 11 Version 25H2 Build 26200
- Microsoft Visual Studio Community 2026 Version 18

History

The history of the Multi-Platform App User Interface (MAUI) is closely tied to the evolution of Xamarin and the introduction of .NET Core.

Xamarin

Founded in 2011, Xamarin aimed to make mobile app development more accessible and efficient by allowing developers to write a single C#/.NET codebase and deploy it across multiple platforms such as iOS and Android.

.NET Core

Released in 2019, .NET Core is an open, cross-platform version of the .NET Framework, enabling developers to run .NET on Windows, Linux, and macOS.

.NET MAUI

Introduced in 2022, .NET MAUI is a cross-platform framework for building native mobile and desktop apps with C# and XAML, evolving from Xamarin.Forms and extending into desktop scenarios.

The transition from Xamarin to .NET MAUI represented a significant architectural shift, integrating the framework into the broader .NET ecosystem and addressing many historical Xamarin challenges. By 2025, .NET MAUI had made substantial progress in desktop support and platform-specific integrations, distinguishing itself from mobile-focused alternatives.

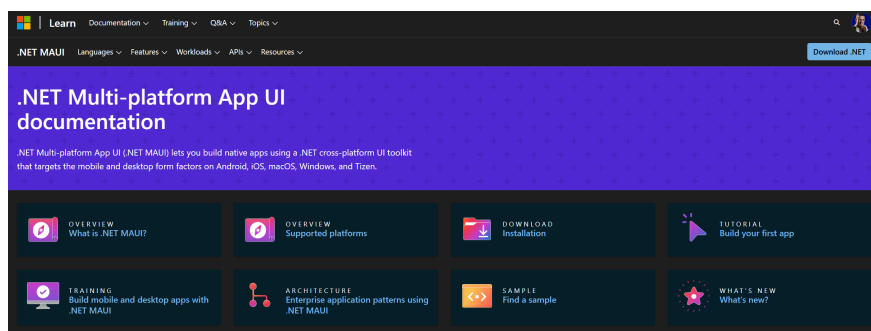


Fig. 4.1. .NET Multi-platform App UI documentation

For more information: [.NET Multi-Platform App User Interface Documentation](#).

Installation

You can download the free Community edition of the integrated development environment, **Microsoft Visual Studio Community Edition**, from the official website: <https://visualstudio.microsoft.com/>.

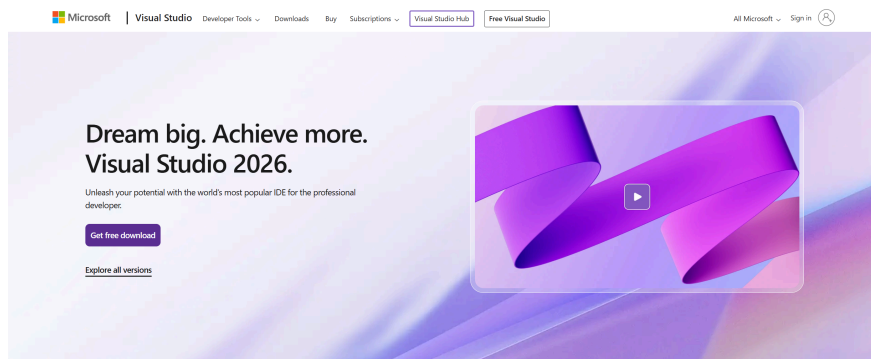


Fig. 4.2. Microsoft Visual Studio 2026 Website

For the development in this chapter we will use **Microsoft Visual Studio Community Edition 2026**; during installation be sure to select the **.NET Multi-platform App UI development workload**/option.

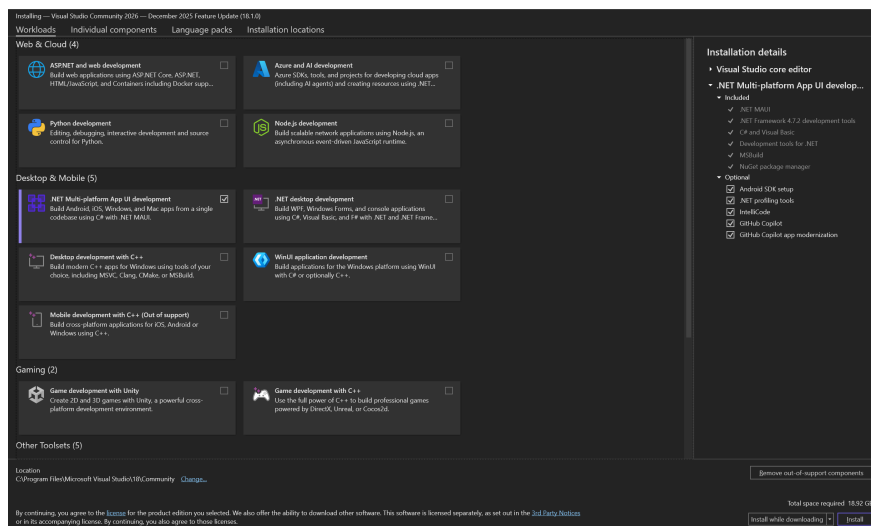


Fig. 4.3. MAUI workload selection during installation

Selecting additional developer tools increases the disk space required to install the IDE.

Proceed with the installation and wait for it to complete; the process may take a considerable amount of time.

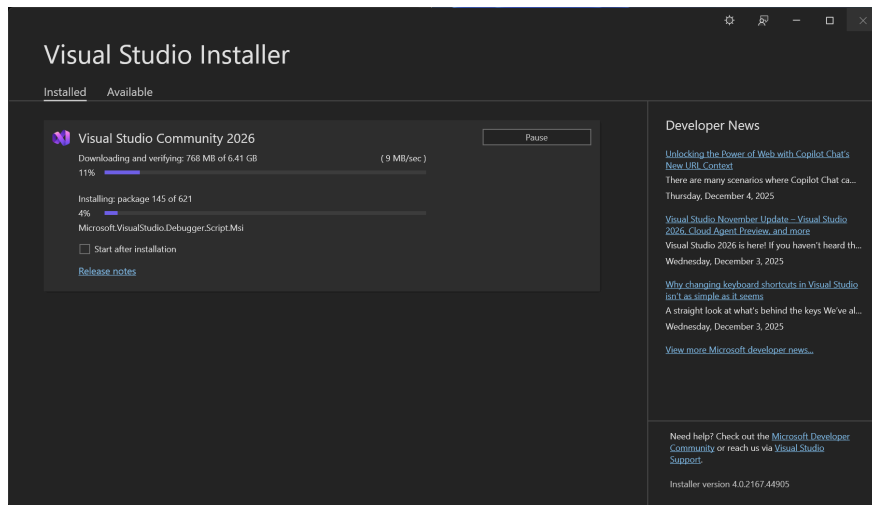


Fig. 4.4. Visual Studio Installation

Applications

Creating any application in the integrated development environment **Microsoft Visual Studio 2026** starts by creating a new project (Fig. 4.5).

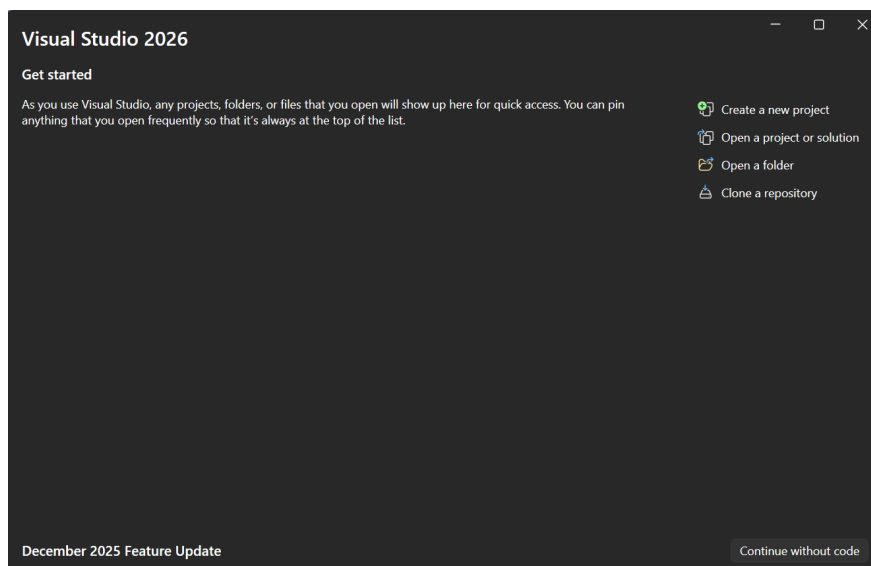
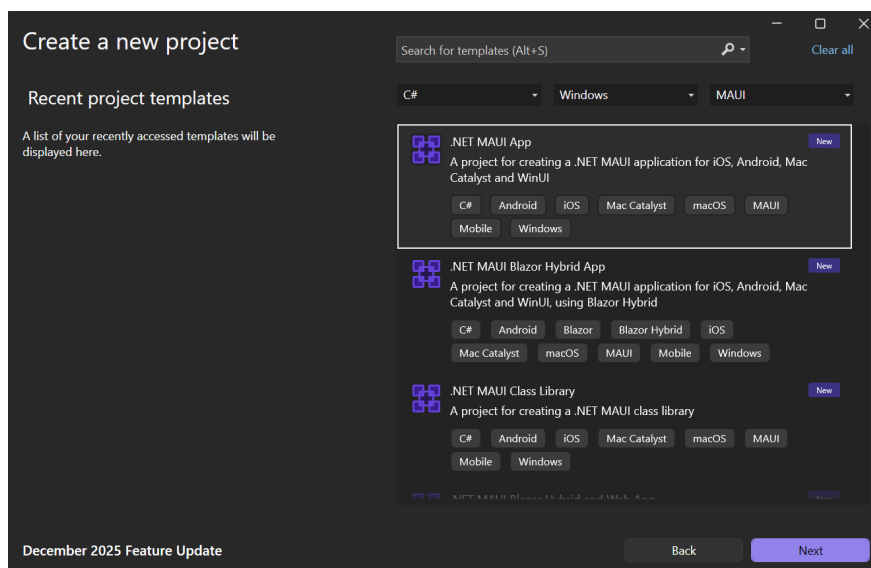


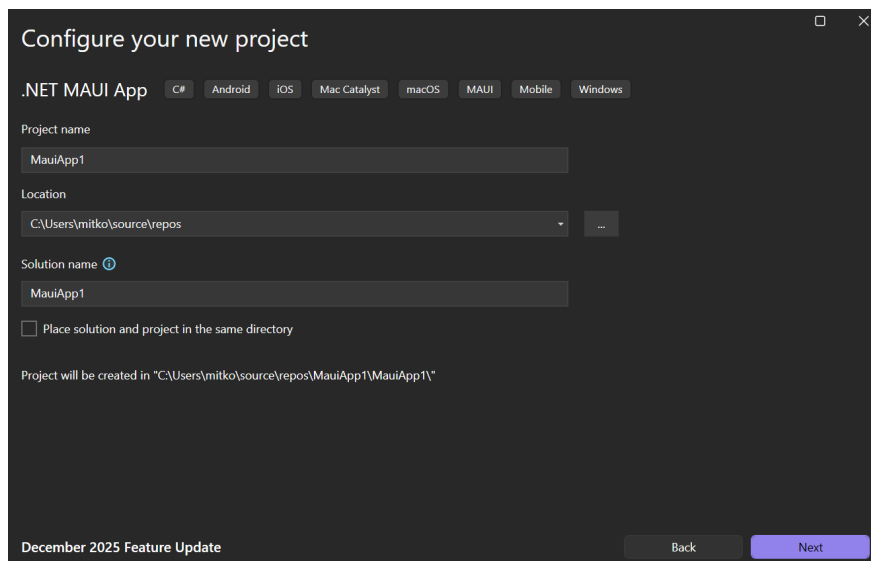
Fig. 4.5. Creating a new project in Visual Studio

На следващия екран е показан избора на на тип на проекта в интегрираната среда за разработка **Microsoft Visual Studio 2026** (Фиг. 4.6).



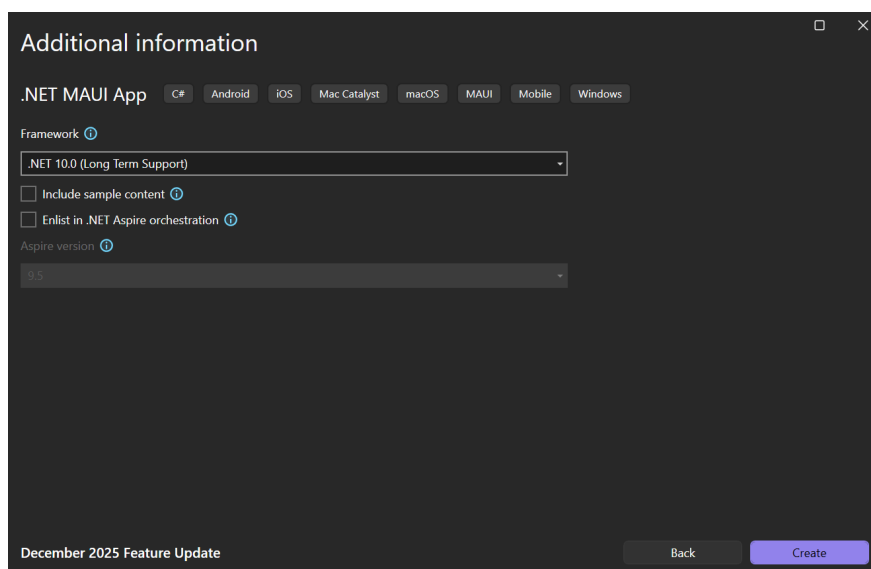
Фиг. 4.6. Избор на тип на проекта в интегрираната среда за разработка Visual Studio

На следващия екран е показан избор на име за проекта и местоположение за съхранението му в интегрираната среда за разработка **Microsoft Visual Studio 2026** (Фиг. 4.7).



Фиг. 4.7. Избор на име за проекта и местоположение за съхранението му в интегрираната среда за разработка Visual Studio

На последният екран е показан избор на версия на .NET (в конкретният пример: *.NET 10 Long Term Support*) в интегрираната среда за разработка **Microsoft Visual Studio 2026** (Фиг. 4.8).



Фиг. 4.8. Избор на име за проекта и местоположение за съхранението му в интегрираната среда за разработка Visual Studio

Suma 4.0

Start the integrated development environment **Microsoft Visual Studio 2026**, create a new project and name it **Suma 4.0**.

MainPage.xaml

This file contains the source code for the application's user interface design.

```
<!-- User Interface (UI): Suma 4.0 -->
<StackLayout Padding="20">

    <!-- Title -->
    <Label Text="Suma 4.0" FontSize="Large" />

    <!-- A -->
    <Label Text="A=" />
    <Entry x:Name="boxA" Keyboard="Numeric" />

    <!-- B -->
    <Label Text="B=" />
    <Entry x:Name="boxB" Keyboard="Numeric" />

    <!-- A+B -->
    <Label Text="A+B=" />
    <Entry x:Name="boxAB" Keyboard="Numeric" />

    <!-- Button -->
    <Button Text="Sum" Clicked="OnButtonClicked" />

</StackLayout>
```

MainPage.xaml.cs

This file contains the source code for the application's business logic.

```
// Business Logic (BL): Suma 4.0
public partial class MainPage : ContentPage
{
    // Constructor
    public MainPage()
    {
        InitializeComponent();
    }

    // Button Click Event Handler
    void OnButtonClicked(object sender, EventArgs args)
    {
        var A = int.Parse(this.boxA.Text);
        var B = int.Parse(this.boxB.Text);
        var C = A + B;
        boxAB.Text = C.ToString();
    }
}
```

Running the app

Run the application and test its functionality as shown in the figure below.

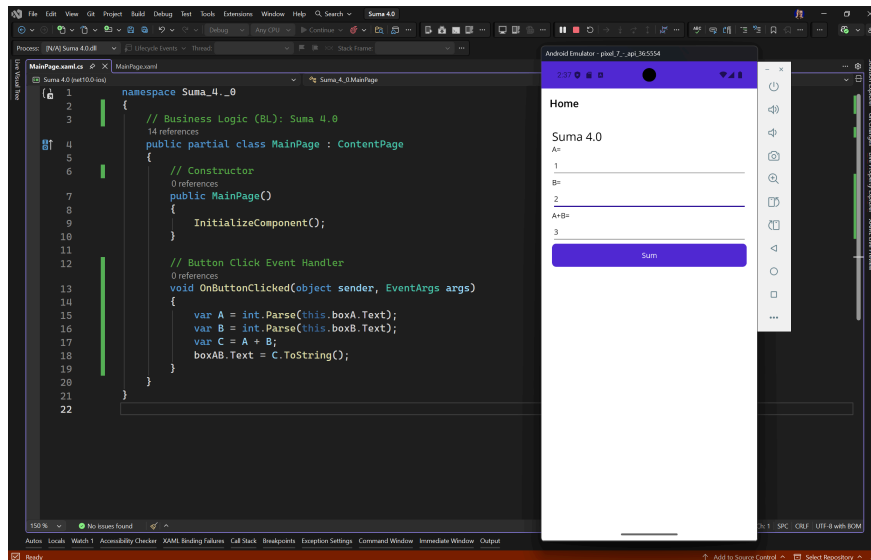


Fig. 4.9. Suma 4.0

Note

Do you notice similarities with the examples from Chapter 2? Try to implement all examples from Chapter 2 yourself, using the **Multi-Platform App User Interface** (MAUI) project type.

Conclusion

- [Note](#)
- [Book](#)
- [Author](#)
- [License](#)
- [Format](#)

Note

Info

How to convert this GitBook e-book into PDF, EPUB and/or MOBI file formats?

Windows

1. Install the following software:

- [node.js](#)
- [Git](#)
- [Calibre](#)

2. Open an elevated command prompt and clone the book repository:

```
git clone https://github.com/dimitarminchev/DCPA.git
```

Result:

```
Cloning into 'DCPA'...
remote: Enumerating objects: 700, done.
remote: Counting objects: 100% (107/107), done.
remote: Compressing objects: 100% (85/85), done.
remote: Total 700 (delta 70), reused 36 (delta 22), pack-reused 593 eceiving objects:
Receiving objects: 100% (700/700), 204.77 MiB | 9.35 MiB/s, done.
Resolving deltas: 100% (410/410), done.
```

3. Enter the newly cloned repository:

```
cd DCPA
```

4. Install the required modules by running the following commands:

```
$env:Path += 'C:\Program Files\Calibre2\'  
npm install -g ebook-convert  
npm install -g gitbook-cli  
gitbook install
```

5. You may encounter an error like this:

```
C:\Users\mitko\AppData\Roaming\npm\node_modules\gitbook-cli\node_modules\npm\node_modu  
    if (cb) cb.apply(this, arguments)  
                ^  
TypeError: cb.apply is not a function  
    at C:\Users\mitko\AppData\Roaming\npm\node_modules\gitbook-cli\node_modules\npm\node_modu  
    at FSReqCallback.oncomplete (node:fs:211:5)
```

To fix this error:

```
cd C:\Users\mitko\AppData\Roaming\npm\node_modules\gitbook-cli\node_modules\npm\  
npm i graceful-fs@4.1.4 --save
```

6. Return to the command prompt and finish the installation:

```
gitbook install
```

7. Check the GitBook version with:

```
gitbook --version
```

The result may look like:

```
CLI version: 2.3.2  
GitBook version: 3.2.3
```

8. Run the procedure to generate the e-book in your chosen format:

- HTML

```
gitbook build
```

A successful run will show output similar to:

```
info: 8 plugins are installed
info: 7 explicitly listed
info: loading plugin "hints"... OK
info: loading plugin "highlight"... OK
info: loading plugin "search"... OK
info: loading plugin "lunr"... OK
info: loading plugin "sharing"... OK
info: loading plugin "fontsettings"... OK
info: loading plugin "theme-default"... OK
info: found 35 pages
info: found 109 asset files
warn: "this.generator" property is deprecated, use "this.output.name" instead
warn: "navigation" property is deprecated
warn: "book" property is deprecated, use "this" directly instead
warn: "options" property is deprecated, use config.get(key) instead
info: >> generation finished with success in 2.2s !
```

- **PDF**

```
gitbook pdf
```

A successful run will show output similar to:

```
info: 8 plugins are installed
info: 7 explicitly listed
info: loading plugin "hints"... OK
info: loading plugin "highlight"... OK
info: loading plugin "search"... OK
info: loading plugin "lunr"... OK
info: loading plugin "sharing"... OK
info: loading plugin "fontsettings"... OK
info: loading plugin "theme-default"... OK
info: found 35 pages
info: found 109 asset files
warn: "this.generator" property is deprecated, use "this.output.name" instead
warn: "navigation" property is deprecated
warn: "book" property is deprecated, use "this" directly instead
warn: "options" property is deprecated, use config.get(key) instead
info: >> generation finished with success in 18.1s !
info: >> 1 file(s) generated
```

- **EPUB**

```
gitbook epub
```

A successful run will show output similar to:

```
info: 8 plugins are installed
info: 7 explicitly listed
info: loading plugin "hints"... OK
info: loading plugin "highlight"... OK
info: loading plugin "search"... OK
info: loading plugin "lunr"... OK
info: loading plugin "sharing"... OK
info: loading plugin "fontsettings"... OK
info: loading plugin "theme-default"... OK
info: found 35 pages
info: found 110 asset files
warn: "this.generator" property is deprecated, use "this.output.name" instead
warn: "navigation" property is deprecated
warn: "book" property is deprecated, use "this" directly instead
warn: "options" property is deprecated, use config.get(key) instead
info: >> generation finished with success in 9.6s !
info: >> 1 file(s) generated
```

- **MOBI**

```
gitbook mobi
```

A successful run will show output similar to:

```
info: 8 plugins are installed
info: 7 explicitly listed
info: loading plugin "hints"... OK
info: loading plugin "highlight"... OK
info: loading plugin "search"... OK
info: loading plugin "lunr"... OK
info: loading plugin "sharing"... OK
info: loading plugin "fontsettings"... OK
info: loading plugin "theme-default"... OK
info: found 35 pages
info: found 111 asset files
warn: "this.generator" property is deprecated, use "this.output.name" instead
warn: "navigation" property is deprecated
warn: "book" property is deprecated, use "this" directly instead
warn: "options" property is deprecated, use config.get(key) instead
info: >> generation finished with success in 8.5s !
info: >> 1 file(s) generated
```

Linux

1. Update your distribution:

```
sudo apt update -y && sudo apt upgrade
```

2. Install the required packages:

```
sudo apt install -y calibre  
sudo apt install -y nodejs  
sudo apt install -y npm  
sudo npm install -g ebook-convert  
sudo npm install -g graceful-fs@4.2.0  
sudo npm install -g gitbook-cli@2.1.2  
gitbook install
```

3. Clone the book repository:

```
git clone https://github.com/dimitarminchev/DCPA.git  
cd DCPA
```

4. Generate the e-books:

```
gitbook pdf  
gitbook mobi  
gitbook epub
```

A successful run will show output similar to:

```
info: 8 plugins are installed
info: 7 explicitly listed
info: loading plugin "hints"... OK
info: loading plugin "highlight"... OK
info: loading plugin "search"... OK
info: loading plugin "lunr"... OK
info: loading plugin "sharing"... OK
info: loading plugin "fontsettings"... OK
info: loading plugin "theme-default"... OK
info: found 36 pages
info: found 80 asset files
warn: "this.generator" property is deprecated, use "this.output.name" instead
warn: "navigation" property is deprecated
warn: "book" property is deprecated, use "this" directly instead
warn: "options" property is deprecated, use config.get(key) instead
info: >> generation finished with success in 34.8s !
info: >> 1 file(s) generated
```

Книга

Dimitar Minchev

Developing Cross Platform Apps

Published
with GitBook



This project **Developing Cross-Platform Apps** presents Microsoft technologies for developing cross-platform applications.

Author

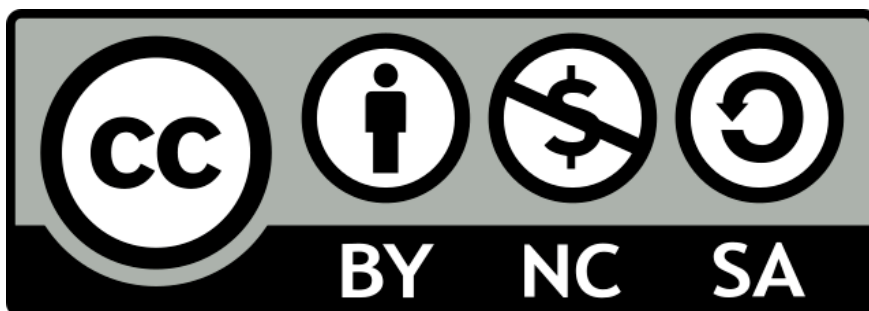
Dimitar Minchev is a university lecturer at the Faculty of Informatics and Technical Sciences at Burgas Free University. He prepares students for the [Republican Student Programming Olympiad](#) through the [Competitive Programming Club](#). He organizes a competition for desktop and mobile application development: [HACKATHON @ BFU](#). He founded the [Programming Talents Academy](#) and the [Robotics School](#) for school students in Burgas. He initiated the student programming contest [CODE@BURGAS](#). He participates in the national program [Training for IT Skills and Career](#) of the Ministry of Education and Science.

Contact
Work: tel. +359 56 900 477 and e-mail: mitko@bfu.bg
Personal: mobile +359 899 148 872 and e-mail: dimitar.minchev@gmail.com
Blog: http://www.minchev.eu

License

Warning

All materials in this e-book are distributed under the [CC-BY-NC-SA](#) license.



You are free to:

Share — copy and redistribute the material in any medium or format.

Adapt — remix, transform, and build upon the material. If you follow the license terms, the licensor cannot revoke these freedoms.

Under the following terms:

Attribution — You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in a way that suggests the licensor endorses you or your use of the material. If you follow the license terms, the licensor cannot revoke these freedoms.

NonCommercial — You may not use the material for commercial purposes.

ShareAlike — If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original.

No additional restrictions — You may not apply legal terms or technological measures that legally restrict others from doing anything the license permits.

Info

The full license text is available online at: [CC-BY-NC-SA](#)

Формат

1. The tutorial is available for free reading as a GitBook-based [electronic version](#).
2. The learning resources are available for free download from the GitHub-based [repository](#).

Format	ISBN	Barcode
PDF	978-619-7126-66-2	 9 786197 126662
MOBI	978-619-7126-67-9	 9 786197 126679
EPUB	978-619-7126-68-6	 9 786197 126686